



UNIVERSITY OF TRENTO

Department of Information Engineering and Computer Science

Bachelor's Degree in
Computer Science

FINAL DISSERTATION

MAINTAINING HITTING SETS IN TEMPORAL HYPERGRAPHS

Supervisor
Alberto Montresor
Co-Supervisor
Quintino Francesco Lotito

Student
Luca Prigione
242880

Academic year 2025/2026

Contents

Abstract	1
1 Introduzione	2
1.1 Definizione del problema	2
1.2 Grafi e Ipergrafi	3
1.3 Ipergrafi temporali	3
1.3.1 Finestra temporale a scorrimento	4
2 Stato dell'arte e lavori correlati	5
2.1 Il problema dell'Hitting Set	5
2.1.1 Definizione formale e varianti	5
2.1.2 Complessità computazionale	6
2.2 Minimum Hitting Set in Ambito Statico	6
2.3 Minimum Hitting Set in Ambito Dinamico	7
2.4 Minimum Hitting Set in Ambito Online	8
3 Definizione del problema e approccio proposto	11
3.1 Definizione formale del problema	11
3.2 Soluzione proposta	12
3.2.1 Intuizione	12
3.2.2 Architettura	12
3.3 Descrizione dell'algoritmo	14
3.3.1 Strutture dati utilizzate	15
3.3.2 Lo Pseudocodice iniziale	15
3.3.3 Analisi Passo-Passo	16
3.3.4 Esempio di Esecuzione	16
3.4 Considerazioni sulla complessità	18
4 Valutazioni sperimentali	20
4.1 Configurazione e ambiente di test	20
4.2 Varianti degli algoritmi analizzati	20
4.3 Istanze di benchmark e caratterizzazione dei dati	21
4.4 Analisi delle prestazioni computazionali	22
4.4.1 Metriche di valutazione e protocollo di tesi	22
4.4.2 Aspettative sperimentali e risultati	22
4.5 Analisi critica e trade-off	24
5 Conclusioni e sviluppi futuri	27
5.1 Sviluppi futuri	27
Bibliografia	27

Abstract

L'aumento nella disponibilità di dati relazionali complessi ha portato negli ultimi anni ad un interesse crescente verso le strutture dati capaci di modellare interazioni di ordine superiore, superando la tradizionale natura binaria dei grafi [3, 2]. Nonostante gli ipergrafi offrano una rappresentazione completa di tali dati relazionali complessi [5], al fine di riuscire ad applicarli in contesti reali è necessaria una visione non statica degli stessi, attraverso l'introduzione del parametro temporale. Questo lavoro affronta il problema combinatorio dell'Hitting Set, uno dei problemi NP-hard introdotti da Karp [16], applicando gli ipergrafi in un contesto temporale e focalizzandosi dunque sul mantenimento incrementale di un insieme di nodi in grado di coprire tutti gli iperarchi attivi all'interno di una finestra temporale a scorrimento. L'obiettivo di questo lavoro è dunque al contempo quello di garantire la validità e l'ottimalità della soluzione proposta. Al fine di garantire la validità della soluzione è necessario offrire una copertura completa dell'Hitting Set al variare del tempo, mentre per garantire il miglior risultato possibile è necessario ottimizzare tre metriche fondamentali: la dimensione della soluzione, il costo computazionale di aggiornamento e la stabilità temporale, definita come la minimizzazione del numero totale di nodi distinti selezionati nel corso dell'intera esecuzione. Per riuscire a garantire una soluzione valida e ottimale è stata progettata un'architettura incrementale articolata in tre processi separati: il processo di rimozione dei nodi dell'Hitting Set, il quale consiste nella rimozione degli iperarchi scaduti e dei nodi non più essenziali, il processo di inserimento dei nuovi nodi, il quale consiste nell'aggiunta dei nuovi iperarchi e nella ricerca di un nuovo Hitting Set, infine nel processo finale di pulizia, il quale consiste in una fase di potatura delle ridondanze incrociate. È stata infine messa alla prova l'efficacia dell'algoritmo, attraverso una campagna sperimentale basata su quattro dataset reali, differenziabili per dimensione, distribuzione e densità. Per ogni caso di test sono state inoltre valutate diverse istanze dell'algoritmo al fine di isolare le componenti più impattanti e significative e permettendo un confronto sui risultati ottenuti. I risultati infine mostrano come la soluzione proposta sia in grado di garantire risultati eccellenti in alcuni contesti caratterizzati da una distribuzione degli arrivi uniforme, riuscendo ad aumentare significativamente il riutilizzo dei nodi e a diminuire la dimensione media della soluzione, con benefici crescenti all'aumentare del rapporto tra ampiezza della finestra e passo di scorrimento. I risultati non sono però completamente positivi, il costo di mantenimento delle strutture dati ausiliarie ed il costo della fase di pulizia finale rendono l'approccio incrementale meno competitivo per non dire completamente inefficiente sul tempo di esecuzione puro in presenza di distribuzioni intermittenti o concentrate, in presenza di tali contesti il ricalcolo da zero dell'Hitting Set può risultare dunque preferibile o, in casi più estremi, l'unica opzione disponibile.

1 Introduzione

La crescente disponibilità di dati relazionali da studiare negli ultimi anni ha portato ad un sempre maggiore interesse nello studio di strutture dati capaci di rappresentare interazioni complesse tra le varie entità. In letteratura, tali tipologie di relazioni erano originariamente gestite attraverso l'utilizzo dei *grafi*, strutture matematiche utilizzate per modellare relazioni uno-a-uno tra più oggetti. Tuttavia, nonostante il loro grande successo, in molti scenari applicativi tale rappresentazione risulta troppo restrittiva, specialmente quando vi è la necessità di considerare interazioni che coinvolgono più entità contemporaneamente. All'interno di numerose applicazioni reali, quali per esempio le comunicazioni presenti all'interno dei social media, così come le interazioni che avvengono all'interno di una transazione bancaria, non è sempre permessa una rappresentazione tramite grafi senza causare una significativa perdita di informazioni. Poiché tale perdita di informazione risulta inevitabile, è stato necessario introdurre una rappresentazione matematica più di alto livello, ovvero gli *ipergrafi*. Tali rappresentazioni matematiche costituiscono una generalizzazione dei grafi classici, in quanto consentono a ciascun iperarco di connettere un numero arbitrario di nodi, rendendoli adatti a modellare interazioni di ordine superiore [2] [5]. Questa tipologia di struttura dati risulta particolarmente utile in tutti quei casi in cui l'obiettivo è rappresentare interazioni complesse senza perdere informazioni strutturali, che i grafi tradizionali tenderebbero invece a scartare. Bisogna inoltre tenere conto che nella maggior parte degli scenari applicativi le interazioni tra entità non rimangono statiche nel tempo, ma evolvono subendo delle variazioni strutturali [14]. Inserendo la variante evolutiva, nuove interazioni vengono create, mentre altre tendono a scomparire, dando vita a un sistema complesso e dinamico che porta una struttura dati teoricamente statica a evolversi nel tempo. Questo tipo di sfida rende dunque necessaria la gestione di tale evoluzione temporale, al fine di poter rappresentare sempre i dati in modo accurato. “Giusto per citare alcuni esempi, la modellazione di grafi temporali e l'analisi delle proprietà temporali possono trovare applicazione nella sociologia e nell'analisi delle reti sociali [...]; nella sicurezza e nel calcolo distribuito [...]; nella biologia” [19]. L'analisi di grandi strutture dinamiche introduce inoltre importanti problemi computazionali [19]. In particolare, è spesso necessario aggiornare continuamente le informazioni di interesse senza poter ricalcolare da capo l'intera soluzione a ogni passo temporale. Di conseguenza, lo studio di tecniche efficienti per il mantenimento di proprietà e strutture combinatorie in ambienti dinamici e temporali svolge un ruolo centrale.

1.1 Definizione del problema

Quando si analizzano sistemi caratterizzati da interazioni collettive, risulta spesso necessario riuscire a trovare un insieme ristretto di elementi che abbia lo scopo di controllare, rappresentare o coprire l'insieme delle interazioni presenti nel sistema. Alcuni tra i problemi più famosi dell'informatica teorica definiti su grafi e ipergrafi si concentrano proprio sulla ricerca di un sottoinsieme minimo di elementi che soddisfi una determinata proprietà. Tra gli esempi più famosi si possono citare il problema del *vertex cover* e quello del *set cover*, problemi di ottimizzazione nei quali l'obiettivo è quello di minimizzare rispettivamente il numero di nodi e di archi necessari a coprire tutti i restanti elementi (nota: ho invertito l'ordine di nodi/archi per farlo corrispondere a vertex/set cover), noti per la loro complessità computazionale, in quanto appartenenti alla classe dei problemi NP-hard [11, 13]. A differenza dei problemi precedenti, in questo lavoro le interazioni non sono statiche ma evolvono nel tempo e inoltre si considerano come strutture dati unicamente gli ipergrafi. Di conseguenza, l'obiettivo non consiste esclusivamente nel determinare una soluzione globalmente ottimale, bensì nel mantenere nel tempo una soluzione valida che possa essere aggiornata in modo efficiente al variare delle interazioni, evitando di doverla ricalcolare completamente a ogni modifica del sistema. Per descrivere in modo più preciso il problema affrontato in questo lavoro consiste nel mantenere un insieme di nodi che riesca a coprire tutte le interazioni rappresentate dagli iperarchi attivi di un ipergrafo temporale. La soluzione deve

quindi adattarsi dinamicamente all'evoluzione della struttura, mantenendo la proprietà di copertura. Oltre alla dimensione media della soluzione, vengono inoltre considerati ulteriori criteri di valutazione. Per avere una soluzione che sia valida non solo in ambito teorico ma anche in ambito pratico, vengono presi in considerazione sia il tempo di esecuzione necessario per aggiornare la soluzione al variare delle interazioni, sia il numero totale di nodi distinti che entrano a far parte dell'insieme selezionato durante l'evoluzione dell'ipergrafo. Tutto ciò per garantire che la soluzione permetta il minor numero di modifiche totali e sia effettivamente utilizzabile anche in contesti reali, dove è necessaria anche la velocità di aggiornamento della soluzione, oltre alla sua correttezza.

1.2 Grafi e Ipergrafi

Quando all'interno del seguente lavoro si trattano i *grafi*, ci si riferisce ad una struttura matematica composta da un insieme di nodi (o vertici) e da un insieme di archi che connettono coppie di nodi. Formalmente, un grafo è una coppia $G = (V, E)$, dove V è l'insieme di nodi e E è l'insieme di coppie di nodi, definita formalmente come:

$$E \subseteq \{\{u, v\} \mid u, v \in V, u \neq v\}.$$

Inoltre, il grado di un nodo v , indicato con $\deg(v)$, rappresenta il numero di archi incidenti su di esso. I grafi si possono poi classificare in base a diverse caratteristiche, come ad esempio l'orientamento degli archi o la presenza di un fattore di peso che può essere associato agli archi o ai nodi. I grafi costituiscono uno strumento fondamentale per la modellazione di sistemi complessi e trovano applicazione nell'analisi delle reti sociali, delle infrastrutture di trasporto, delle reti di comunicazione e dei sistemi biologici [23]. Al contempo, però, tali strutture dati possono risultare estremamente limitanti quando si tratta di rappresentare, senza alcuna perdita di informazione, interazioni che coinvolgono molteplici entità contemporaneamente.

Gli *ipergrafi* sono invece la generalizzazione dei grafi; tali strutture dati complesse sono composte da un insieme di nodi e da un insieme di iperarchi, all'interno dei quale ogni iperarco può connettere un numero arbitrario di nodi. Formalmente, un ipergrafo è una coppia $H = (V, E)$, dove V è l'insieme di nodi e E è l'insieme di sottoinsiemi di V , definito formalmente come:

$$E \subseteq \{e \subseteq V \mid e \neq \emptyset\}.$$

La definizione di grado non cambia particolarmente per gli ipergrafi; tuttavia, introducendo la cardinalità di un iperarco, indicata con $|e|$, si rappresenta il numero di nodi coinvolti nell'interazione descritta dall'iperarco [5]. Come per i grafi, anche gli ipergrafi possono essere classificati in base a diverse caratteristiche, come ad esempio l'orientamento degli iperarchi o la presenza di un fattore di peso che può essere associato agli iperarchi o ai nodi. Tuttavia, all'interno di questo lavoro verranno considerati solamente ipergrafi non orientati e senza peso. Gli ipergrafi, a differenza dei grafi, consentono di rappresentare senza perdita di informazione interazioni che coinvolgono molteplici entità contemporaneamente. Tale generalizzazione permette di rappresentare sistemi caratterizzati da interazioni collettive [3, 2].

1.3 Ipergrafi temporali

Come descritto in precedenza, gli ipergrafi permettono di rappresentare interazioni che coinvolgono più entità contemporaneamente, ma la formulazione classica della struttura dati riesce a descrivere soltanto la struttura statica del sistema. Tale rappresentazione risulta però limitata nel momento in cui entrano in considerazione contesti reali e dinamici, come le reti sociali: le relazioni tra individui evolvono nel tempo, dunque una rappresentazione statica del sistema non risulta sufficiente a catturare la complessità dello stesso. Per questo motivo, è necessario estendere il concetto di ipergrafo integrando la dimensione temporale. Tale integrazione genera una struttura dati che prende il nome di *ipergrafo temporale*, la quale permette di rappresentare simultaneamente la natura collettiva delle interazioni e la sua evoluzione nel tempo [12]. Dal punto di vista formale, un ipergrafo temporale è una coppia $H_t = (V, E_t)$, all'interno della quale V è l'insieme dei nodi e E_t è l'insieme delle coppie (E, t) , definito

come:

$$E_T \subseteq \{(E, t) \mid E \subseteq V, E \neq \emptyset, t \in T\}.$$

All'interno di E_T ogni elemento (E, t) rappresenta un'interazione che coinvolge simultaneamente tutti i nodi presenti all'interno dell'iperarco E all'istante t . All'interno del seguente lavoro l'informazione temporale viene associata a un singolo istante di tempo t , però tale informazione può essere associata anche ad un intervallo di tempo. L'introduzione della componente temporale viene dunque utilizzata per distinguere sistemi che, pur condividendo la stessa struttura aggregata, differiscono nell'ordine delle interazioni, un aspetto spesso determinante nello studio di processi dinamici, all'interno dei quali la sequenza degli eventi può influenzare significativamente l'evoluzione del sistema [14].

1.3.1 Finestra temporale a scorrimento

All'interno dei sistemi applicativi, l'analisi in modo ripetitivo di tutte le interazioni presenti dall'inizio del periodo di osservazione porta spesso a rappresentazioni poco significative della realtà. Relazioni più recenti devono spesso risultare più rilevanti rispetto a relazioni avvenute in passato e aggregare il tutto indiscriminatamente rischia di appiattire informazioni temporali importanti. Per ovviare a questo problema è stata introdotta l'idea di *finestra temporale a scorrimento*. Data una dimensione della finestra temporale ΔT , ogni interazione osservata all'istante t viene considerata valida solamente se compresa all'interno dell'intervallo temporale $[t, t + \Delta T]$, trascorso il quale, essa non concorre più alla rappresentazione del sistema. In altre parole, se ci troviamo in un generico istante t_c , l'ipergrafo deve risultare costituito esclusivamente dagli iperarchi corrispondenti a eventi verificatisi nell'intervallo $[t_c - \Delta T, t_c]$. L'utilità di questo approccio è immediata: durante l'analisi di un social network, ad esempio, potrebbe risultare sensato analizzare le conversazioni degli ultimi sette giorni piuttosto che quelle presenti dalla nascita di tale social network, mentre in un contesto aziendale, si potrebbe preferire analizzare gli eventi avvenuti nella durata di un mese piuttosto che quelli dell'intero anno. Una caratteristica fondamentale è la possibilità da parte della finestra di continuare a scorrere lungo l'asse temporale con un passo di avanzamento δ , differente rispetto all'ampiezza ΔT della finestra temporale, per fare in modo che rimanga aggiornata con lo scorrere del tempo. Ad ogni spostamento viene costruita una nuova istanza dell'ipergrafo, contenente esclusivamente le interazioni comprese nella finestra corrente.

2 Stato dell'arte e lavori correlati

Durante la definizione del problema (capitolo 1.1) abbiamo evidenziato come il presente lavoro affronti uno specifico sottoproblema riconducibile ad uno dei più noti problemi NP-hard [16], senza tuttavia approfondire effettivamente la definizione formale né le motivazioni che ne giustificano l'interesse. All'interno del seguente capitolo dunque tratteremo in modo più rigoroso l'introduzione formale al problema nelle sue varianti principali, analizzandone la complessità computazionale e tracciando l'evoluzione dello stato dell'arte: dai modelli statici tradizionali fino alle moderne formulazioni on-line su flussi di dati, che costituiranno il riferimento per gli sviluppi successivi dell'elaborato.

2.1 Il problema dell'Hitting Set

Il problema dell'Hitting Set è uno dei pilastri dell'ottimizzazione combinatoria e della teoria della complessità computazionale, noto storicamente per essere uno dei 21 problemi NP-completi di Karp [16]. Tale problema riveste una particolare importanza pratica in quanto trova numerose applicazioni in contesti dell'informatica, bioinformatica, data mining e all'interno dei sistemi di diagnostica [21, 20]. Dal punto di vista teorico, il problema dell'Hitting Set è strettamente correlato al problema noto come *Set Cover*, di cui rappresenta il duale. Per tale motivo, alcune soluzioni proposte per il Set Cover verranno citate in seguito nell'ambito dei lavori correlati.

2.1.1 Definizione formale e varianti

Sia U un insieme finito, detto universo, e sia $S = \{S_1, S_2, \dots, S_m\}$ una famiglia di sottoinsiemi di U . Un sottoinsieme $H \subseteq U$ è definito *Hitting Set* per S se interseca ogni sottoinsieme appartenente a S , ovvero se

$$\forall S_i \in S, H \cap S_i \neq \emptyset.$$

In altre parole, ogni sottoinsieme della collezione, nel nostro caso dunque ogni iperarco, deve contenere almeno un elemento appartenente ad H [11, 5]. La condizione che rende però tale problema computazionalmente complesso da risolvere non risiede nella difficoltà che si può incontrare nel trovare uno di questi sottoinsiemi, bensì in quella di trovare il sottoinsieme con cardinalità minima che rispetti le proprietà precedentemente citate.

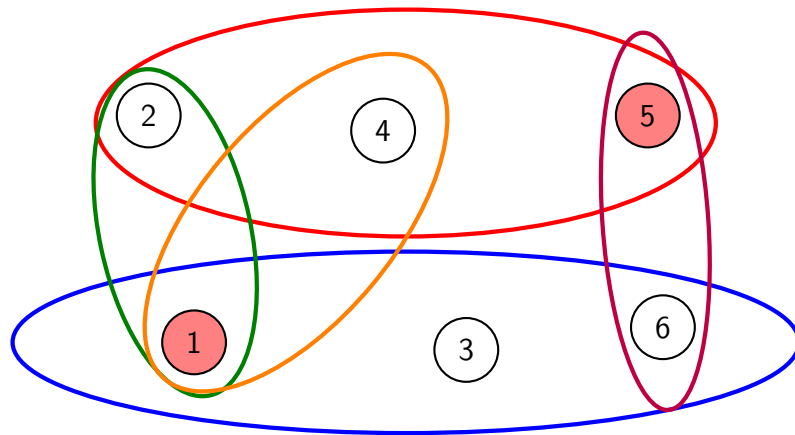


Figure 2.1: Esempio di Hitting Set in un ipergrafo.

Da ora in avanti risulterà inoltre importante distinguere tra hitting set minimale (o Minimal Hitting Set) e hitting set di cardinalità minima (da ora in poi citato come Minimum Hitting Set o MHS): un hitting set minimo è necessariamente minimale, ma non viceversa. Tale puntualizzazione è importante in quanto il problema di trovare un hitting set di cardinalità minima è NP-hard, mentre l'enumerazione

di tutti i Minimal Hitting Set è un problema la cui complessità è legata alla struttura dell'ipergrafo. All'interno dell'ambiente accademico sono state analizzate diverse varianti del problema, di seguito riporto le più discusse [10]:

- **Offline Hitting Set:** variante classica del problema nella quale l'intera famiglia di insiemi è nota a priori e rimane invariata durante l'esecuzione dell'algoritmo.
- **Weighted Hitting Set:** variante nella quale ciascun elemento dell'universo ha un peso o un costo associato; l'obiettivo consiste nel determinare un hitting set che minimizzi il costo totale anziché la sola cardinalità.
- **k -Hitting Set:** variante in cui la cardinalità di ciascun insieme della collezione è limitata superiormente da una costante k .
- **Enumeration of Minimal Hitting Sets:** variante che richiede la generazione di tutti gli hitting set minimali della famiglia di insiemi considerata.
- **Dynamic Hitting Set:** variante che permette alla collezione di insiemi di venire modificata nel tempo attraverso operazioni di inserimento e rimozione, rendendo necessario l'aggiornamento delle soluzioni calcolate.
- **Online Hitting Set:** variante che permette alla collezione di insiemi di venire modificata progressivamente e che obbliga a prendere decisioni senza la possibilità di conoscere gli input futuri.

Tra le varianti sopra elencate, particolare attenzione sarà riservata a quelle che introducono una dimensione temporale nella gestione dell'Hitting Set, vale a dire le varianti offline, dinamica e online, che rappresentano il principale oggetto di studio del presente lavoro.

2.1.2 Complessità computazionale

La natura computazionale dell'Hitting Set è intrinsecamente complessa. Nella celebre opera di Richard Karp del 1972, il problema dell'Hitting Set venne inserito tra i 21 problemi originari dimostrati NP-completi (nella sua versione decisionale) [16]. Di conseguenza, la variante di ottimizzazione (MHS) appartiene alla classe di complessità NP-hard. Le implicazioni di questa classificazione sulla letteratura scientifica sono profonde: sotto l'ipotesi $P \neq NP$, il Minimum Hitting Set (MHS) non può essere approssimato in tempo polinomiale entro un fattore logaritmico rispetto alla dimensione dell'universo [15, 9]. Per definire il tutto più formalmente, il problema non è approssimabile entro un fattore $(1 - o(1)) \ln m$, dove $m = |S|$ [9]. A causa dell'intrattabilità nel caso peggiorativo di questo problema, la ricerca si è focalizzata sullo sviluppo di algoritmi euristici, algoritmi parametrizzati e su modelli temporali in grado di aggiornare le soluzioni sfruttando la coerenza durante lo sviluppo dei dati senza dover rieseguire il calcolo da zero [8].

2.2 Minimum Hitting Set in Ambito Statico

Nel contesto statico l'istanza del problema è conosciuta a priori nella sua interezza e non varia nel tempo. Gli algoritmi lavorano quindi all'interno di un'istanza immutabile e hanno il solo scopo di calcolare il Minimum Hitting Set (MHS), o di avvicinarsi quanto più possibile alla soluzione ottimale. La letteratura scientifica legata all'ambito statico si è sviluppata lungo tre filoni principali: gli algoritmi esatti, il calcolo delle euristiche di ottimizzazione e infine gli algoritmi parametrizzati [21, 15, 8].

Algoritmi esatti

Il filone degli algoritmi esatti comprende tutte le soluzioni in grado di fornire sempre una soluzione ottimale al problema. Tale capacità di risoluzione comporta tuttavia un aumento nel costo computazionale necessario al calcolo della soluzione, che nella maggior parte delle applicazioni reali risulta proibitivo. Gli approcci algoritmici più utilizzati all'interno di questo filone sono gli algoritmi basati su branch-and-bound e sulle tecniche di dualizzazione degli ipergrafi [22]. Tali algoritmi consistono nell'esplorare sistematicamente le soluzioni eliminando tutte le configurazioni che non possono portare

a risultati migliori, fino a determinare il Minimum Hitting Set (MHS) [4]. Linee di ricerca più recenti consistono nel formulare il problema come un'istanza di soddisfacibilità booleana (SAT) o di programmazione lineare intera (ILP). In queste formulazioni, a ciascun elemento dell'universo viene associata una variabile decisionale, mentre opportuni vincoli garantiscono che ogni insieme sia colpito da almeno un elemento selezionato. Uno degli approcci più rilevanti in questo contesto introduce il paradigma dell'*Implicit Hitting Set*, introdotto da Moreno-Centeno e Karp [21], attraverso il quale la costruzione della soluzione avviene iterativamente combinando la generazione di candidati con la verifica dei vincoli.

Euristiche e algoritmi approssimati

Gli algoritmi euristici e approssimati permettono di superare i limiti computazionali posti dagli approcci esatti a discapito della precisione della soluzione fornita. La natura NP-Hard del problema dell'Hitting Set [16], ha portato una parte significativa della letteratura a concentrarsi sullo sviluppo di algoritmi capaci di produrre soluzioni di buona qualità e non più ottime in tempi inferiori rispetto agli approcci esatti. Tra gli approcci più diffusi vi sono gli algoritmi greedy, originariamente studiati nel contesto del Set Cover Problem da Johnson [15] e successivamente analizzati da Chvátal [7]. Tali implementazioni greedy possono essere classificate in due famiglie principali: le euristiche *vertex-oriented* e le euristiche *hyperedge-oriented*; tali famiglie si distinguono in base alla tipologia di scelta greedy effettuata, da una parte si privilegia il contributo globale alla copertura, mentre dall'altra si privilegia una scelta locale in base all'iperarco analizzato. Tuttavia, sebbene gli algoritmi greedy siano efficienti, la soluzione ottenuta non sempre soddisfa i criteri di minimalità per inclusione richiesti dal Minimum Hitting Set (MHS). Per ovviare a questo limite, vengono spesso applicate procedure di post-processing volte a eliminare gli elementi ridondanti della soluzione. Oltre agli algoritmi greedy e alle procedure di post-processing, la letteratura ha proposto anche approcci metaeuristici basati dunque su algoritmi evolutivi e su tecniche di ricerca locale, i quali sono stati particolarmente impiegati in applicazioni di grandi dimensioni. Tuttavia, tali metodologie non forniscono garanzie teoriche sulla qualità della soluzione e vengono valutate prevalentemente attraverso analisi sperimentali.

Algoritmi parametrizzati

L'utilizzo di algoritmi parametrizzati infine permette di raggiungere una soluzione ottimale al problema in tempi computazionalmente accettabili limitando però la generalità dell'algoritmo risolutivo. Tale filone algoritmico si specializza infatti nell'analisi del problema basandosi su assunzioni riguardo a parametri specifici dell'istanza, quali la cardinalità della soluzione cercata o la dimensione massima degli insiemi. Queste assunzioni permettono di abbattere i costi computazionali delle soluzioni fornite, precludendo tuttavia una risoluzione generale del problema. In presenza di parametri sufficientemente piccoli, tali tecniche consentono di ottenere algoritmi Fixed-Parameter Tractable (FPT) [8], rendendo risolvibili in tempi pratici alcune classi di istanze altrimenti proibitive.

Considerazioni finali

Gli approcci presentati assumono che l'istanza del problema sia nota e immutabile, un'ipotesi spesso irrealistica all'interno degli scenari reali. Questa limitazione motiva lo studio del problema in contesto dinamico, nel quale insiemi e universo possono variare nel tempo.

2.3 Minimum Hitting Set in Ambito Dinamico

Quando si tratta di problemi in ambito dinamico, l'istanza del problema non è conosciuta a priori, come visto nel precedente contesto statico, ma la struttura si evolve nel tempo attraverso modifiche apportate all'insieme degli iperarchi che compongono l'ipergrafo. All'interno di tali scenari, la possibilità di un ricalcolo completo del Minimum Hitting Set (MHS) a seguito di ogni aggiornamento risulta essere computazionalmente proibitivo; l'architettura del sistema ci impone dunque di adottare strategie differenti. L'obiettivo degli algoritmi dinamici risulta quindi quello di riuscire a mantenere una soluzione valida durante l'intera esistenza del sistema, avendo però la possibilità di aggiornare tale soluzione ogni qualvolta avvenga una modifica alla struttura dell'ipergrafo, cercando al contempo di evitare un ricalcolo completo del Minimum Hitting Set (MHS). La letteratura scientifica relativa a tali problemi può essere suddivisa in tre filoni principali: approcci basati su ricerca locale, approcci

basati su strutture dati ausiliarie e approcci con garanzie di approssimazione [1].

Algoritmi di ricerca locale

Gli algoritmi di ricerca locale affrontano il problema mantenendo una soluzione corrente e applicando, ad ogni aggiornamento dell'ipergrafo, il minimo numero di modifiche locali necessarie a ripristinare la validità e la minimalità dell' hitting set. Il loro obiettivo rimane dunque quello di mantenere una soluzione più vicina possibile a quella ottimale senza però dover procedere a una ristrutturazione globale dell'istanza. Uno dei principi fondanti di questa classe di algoritmi è la gestione differenziata delle operazioni di *inserimento* e di *cancellazione* di un iperarco I , in quanto per ciascuna operazione è necessario apportare in modo distinto delle modifiche all' hitting set precedentemente selezionato. Gli approcci si distinguono per la semplicità implementativa e per il basso costo per operazione nei casi medi, sebbene spesso presentino solo garanzie di complessità *ammortizzate* piuttosto che *worst-case*.

Algoritmi basati su strutture dati ausiliarie

Un secondo filone di ricerca sfrutta strutture dati ausiliarie per tenere traccia in modo efficiente delle dipendenze tra vertici e iperarchi, riducendo il costo degli aggiornamenti. L'idea di fondo consiste nell'associare a ciascun iperarco un'informazione di copertura, ad esempio il numero di vertici della soluzione corrente che lo intersecano, e a ciascun vertice della soluzione gli iperarchi per i quali esso risulta indispensabile, ovvero quelli non coperti da alcun altro elemento della soluzione. Mantenere queste informazioni aggiornate permette di localizzare rapidamente le porzioni della soluzione da modificare a seguito di un inserimento o di una cancellazione, evitando una scansione completa dell'istanza ad ogni operazione. Un approccio diverso, che agisce direttamente sull'ipergrafo duale, consiste nell'indicizzare i trasversali minimali e propagare su di essi gli aggiornamenti in modo incrementale, sfruttando la corrispondenza tra Minimum Hitting Set (MHS) e ipergrafo duale [22]. Tali soluzioni, nonostante siano tra le migliori dal punto di vista della precisione e dell'efficienza computazionale, presentano serie limitazioni dal punto di vista dell'overhead di memoria richiesto dalle strutture ausiliarie, che cresce con la dimensione e la densità dell'ipergrafo.

Algoritmi con garanzie di approssimazione

Il terzo filone rinuncia infine alla minimalità esatta in favore di soluzioni approssimate, ottenendo in cambio migliori garanzie asintotiche all'interno dei modelli *completamente dinamici*, ovvero in presenza allo stesso tempo sia di inserimenti sia di cancellazioni. In questo quadro si inseriscono gli algoritmi per Minimum Hitting Set (MHS) dinamico con fattore di approssimazione $O(\log n)$, analoghi ai risultati noti per il problema del Set Cover dinamico [1]. L'approccio generale consiste nel tollerare una soluzione di dimensione superiore all'ottimo di un fattore logaritmico, garantendo però che ogni aggiornamento venga gestito tramite ammortizzazione in tempo polinomiale. Queste tipologie di soluzioni risultano particolarmente rilevanti quando la dimensione dell'ipergrafo è elevata e la minimalità stretta della soluzione è sacrificabile in favore dell'efficienza computazionale, come spesso accade nei sistemi reali in cui l'efficienza può essere considerata più importante di una soluzione cardinalmente ottima. Tuttavia, per tutte le applicazioni nelle quali la qualità della soluzione è il requisito primario, gli approcci approssimati risultano insufficienti e inadatti.

Considerazioni finali

Gli algoritmi dinamici descritti presentano però un limite intrinseco: operano su un ipergrafo il cui stato corrente è sempre interamente disponibile. In numerose applicazioni reali, tuttavia, gli aggiornamenti arrivano in sequenza e le decisioni devono essere prese *irrevocabilmente* al momento della ricezione, senza conoscenza degli eventi futuri.

2.4 Minimum Hitting Set in Ambito Online

In numerosi scenari applicativi, assumere che l'intera istanza del problema sia disponibile prima dell'esecuzione dell'algoritmo risulta poco realistico. Questa limitazione è alla base del contesto online: nei sistemi reali le informazioni vengono osservate con il passare del tempo e, come tali, le decisioni devono essere prese senza avere la possibilità di conoscere gli aggiornamenti futuri e dovendo impiegare il minor tempo possibile. I problemi analizzati in ambito online non hanno possibilità di conoscere

gli aggiornamenti futuri in quanto l'istanza viene rivelata in maniera incrementale. Ogni qualvolta arrivi una sequenza di iperarchi, l'algoritmo deve riuscire a verificare in modo corretto se e come serve modificare l'Hitting Set, in base alla copertura fornita dall'Hitting Set precedente e dalla durata degli iperarchi che lo stesso sta già coprendo. Una delle caratteristiche principali che differenzia il modello online rispetto a quello dinamico è l'irrevocabilità delle decisioni. Una volta che un nodo viene selezionato all'interno della soluzione, questo non deve più essere rimosso nelle fasi successive. Questo vincolo apparentemente innocuo introduce una componente asimmetrica tra le operazioni disponibili e rende necessario bilanciare la qualità della soluzione corrente con l'incertezza riguardante l'evoluzione futura dell'istanza. A riguardo, la letteratura si distingue principalmente tra gli algoritmi deterministici e quelli randomizzati, che adesso andremo ad analizzare.

Algoritmi deterministici

Con il termine algoritmi deterministici si intendono tutti quegli algoritmi che affrontano il problema del Minimum Hitting Set Online seguendo regole fisse e prive di componenti randomiche. L'assenza di casualità rende tali approcci particolarmente semplici da analizzare dal punto di vista teorico e facilmente implementabili in sistemi reali, poiché il comportamento dell'algoritmo è prevedibile e riproducibile. Le strategie più comuni operano secondo criteri *greedy*, ovvero applicando algoritmi che prendono decisioni di ottimizzazione locale con la speranza di arrivare al contempo a una minimizzazione globale della soluzione del problema. Attraverso lo studio di un problema duale dell'Hitting Set Problem in ambito Online, ovvero il Set Cover Problem definito in ambito Online, i ricercatori Alon, Awerbuch e Azar hanno introdotto e dimostrato che nessun algoritmo deterministico può ottenere un competitive ratio migliore di $\Omega(\log m \log n)$, dove m è il numero di iperarchi e n il numero di vertici dell'istanza, fornendo al contempo un algoritmo $O(\log m \log n)$ -competitivo che rende tale limite quasi ottimo [24]. La letteratura definisce però anche l'esistenza di limiti teorici sulle prestazioni, espressi in termini di *competitive ratio*, ottenibili dagli stessi. Tali limiti dipendono dall'istanza utilizzata, ovvero dal numero di nodi, di iperarchi o dalla frequenza massima, scelta come riferimento per l'analisi effettuata.

Algoritmi randomizzati

Per superare le barriere computazionali poste dai lower bound degli algoritmi deterministici, la letteratura introduce la randomizzazione. Tali algoritmi probabilistici si distinguono poiché non scelgono i vertici in modo univoco, ma si basano su una distribuzione di probabilità attraverso la quale definiscono con quali nodi coprire l'iperarco corrente. L'idea alla base consiste nello sfruttare la casualità per rendere il comportamento dell'algoritmo meno prevedibile rispetto alla sequenza di input osservata. Il vantaggio teorico della randomizzazione non può però manifestarsi in modo uniforme su tutte le varianti del problema, ma deve dipendere fortemente dalle parametrizzazioni considerate. Per esempio, quando l'analisi viene condotta in funzione della frequenza d dell'istanza, l'utilizzo della randomizzazione produce un miglioramento sostanziale: contro il limite deterministico $\Theta(d)$ discusso in precedenza, un approccio basato su un arrotondamento (*rounding*) probabilistico della soluzione frazionaria, consente di ottenere un competitive ratio atteso $O(\log d)$, portando a un miglioramento esponenziale rispetto al caso deterministico [6]. Questo risultato non è però generalizzabile ad ogni formulazione del problema. Nel caso generale dell'Online Set Cover, parametrizzato rispetto al numero di vertici n e di iperarchi m , Feige e Korman sono riusciti a dimostrare che, sotto ragionevoli ipotesi di complessità computazionale, nessun algoritmo randomizzato polinomiale può ottenere un competitive ratio migliore di $\Omega(\log m \log n)$, pareggiando di fatto il limite deterministico in questo regime [17]. Possiamo affermare dunque che la randomizzazione non permette di garantire un vantaggio assoluto sugli algoritmi deterministici, dato che la sua efficacia è strettamente legata al parametro rispetto al quale si conduce l'analisi e alla particolare struttura combinatoria o geometrica dell'istanza considerata. Tra le tecniche più utilizzate sono presenti approcci basati su selezione probabilistica dei candidati, campionamenti incrementali e strategie che associano pesi dinamici ai vertici dell'ipergrafo, aggiornandoli nel tempo. Quando si tratta di soluzioni basate su algoritmi randomizzati la valutazione delle prestazioni di tali algoritmi viene effettuata considerando il *competitive ratio atteso*, ossia il rapporto tra il costo medio della soluzione online e quello dell'ottimo offline calcolato sull'intera sequenza. Poiché gli algoritmi randomizzati introducono una maggiore complessità analitica e una minore prevedibilità operativa, risulta spesso necessario bilanciare la qualità attesa della soluzione e la stabilità del

comportamento nei contesti applicativi reali.

Considerazioni finali

Il modello online rappresenta un'estensione naturale del problema del Minimum Hitting Set (MHS), in quanto ci permette di studiare il problema all'interno di ambienti nei quali l'istanza non è interamente disponibile e le decisioni devono essere prese in modo incrementale e irrevocabile. All'interno dell'ambito online, a causa della complessità del problema stesso, le soluzioni proposte dagli algoritmi vengono analizzate tramite il *competitive ratio*, strumento che consente di confrontare il costo della soluzione proposta con il corrispettivo risultato ottimale.

3 Definizione del problema e approccio proposto

3.1 Definizione formale del problema

Il problema proposto si colloca all'interno di un contesto dinamico, dove le relazioni tra le entità (rappresentate come sottoinsiemi di vertici) si manifestano nel tempo e mantengono la loro rilevanza solo per un intervallo limitato, decorso il quale decadono. L'obiettivo è quello di mantenere un Hitting Set (si veda il Capitolo 2.1) in grado di coprire l'insieme completo degli iperarchi. La soluzione deve essere aggiornata in modo incrementale ad ogni movimento della finestra temporale, minimizzando il costo computazionale ed evitando un ricalcolo completo della soluzione a ogni shift. In termini formali, stiamo trattando una variante online del problema del *Temporal Hitting Set* definita in un contesto di *sliding window*.

Modello

Per descrivere al meglio l'intuizione, modelliamo l'input come un ipergrafo temporale $H = (V, E)$, dove ogni iperarco (S, t) consiste in un sottoinsieme di vertici $S \subseteq V$ in un istante temporale $t \in \mathbb{T}$, dove $\mathbb{T}$ rappresenta il dominio temporale in $\mathbb{R}_{\geq 0}$, che definisce il tempo di arrivo di tale iperarco. Fissata un'ampiezza di finestra ϵ grande a piacere, definiamo l'insieme degli iperarchi attivi al tempo t come:

$$W(t) = \{(S, t') \in E \mid t - \epsilon \leq t' \leq t\}.$$

Un iperarco (S, t') è quindi attivo nell'intervallo $[t - \epsilon, t]$, dopo il quale *decade* ed esce dalla finestra. L'avanzamento della finestra si basa su dei **passi temporali fissi e prefissati** di ampiezza $\delta > 0$, rendendo il sistema indipendentemente da quando e quanti iperarchi arrivano nel frattempo. Definiamo quindi gli *istanti di osservazione*

$$\tau_k = k \cdot \delta, \quad k = 0, 1, 2, \dots$$

In corrispondenza di ciascun τ_k , l'algoritmo deve produrre un *hitting set* $X(\tau_k) \subseteq V$ ammissibile per $W(\tau_k)$, cioè tale che

$$\forall (S, t') \in W(\tau_k), \quad X(\tau_k) \cap S \neq \emptyset,$$

tenendo conto di tutti gli iperarchi arrivati nell'intervallo (τ_{k-1}, τ_k) e di quelli scaduti perché $t' < \tau_k - \epsilon$, e aggiornando dunque la soluzione utilizzata al tempo $X(\tau_k)$ a partire da $X(\tau_{k-1})$, ponendo come condizione iniziale $X(\tau_0) = \emptyset$.

Obiettivo

L'algoritmo in questione deve produrre, per ogni istante di osservazione τ_k un insieme $X(\tau_k) \subseteq V$ tale che $X(\tau_k)$ sia un hitting set ammissibile per la finestra attiva $W(\tau_k)$. Tra tutte le soluzioni ammissibili, si desidera ottimizzare i seguenti criteri:

- **Dimensione della soluzione:** minimizzare la cardinalità della soluzione $|X(\tau_k)|$, in modo da mantenere minimo il numero di vertici selezionati ad ogni istante temporale.
- **Costo di aggiornamento:** minimizzare il numero di vertici distinti selezionati nel corso dell'intera esecuzione, $R = |\bigcup_k X(\tau_k)|$, così da aumentare il riutilizzo dei nodi della soluzione nel tempo ed evitare cambiamenti eccessivamente frequenti nei vertici utilizzati.
- **Costo computazionale:** minimizzare il tempo di aggiornamento c_k ossia il tempo richiesto per ottenere $X(\tau_k)$ a partire dall'informazione disponibile al passo precedente, mantenendo il

calcolo compatibile con il ritmo di avanzamento della finestra. Questo ci permette, di riflesso, di minimizzare il tempo di esecuzione totale del nostro algoritmo.

L'obiettivo completo del problema è dunque quello di mantenere nel tempo una sequenza di *Hitting Set* ammissibili che siano simultaneamente compatti, stabili ed efficienti da aggiornare in presenza dell'inserimento e della scadenza degli iperarchi.

3.2 Soluzione proposta

Una volta definito in modo formale il problema, questa sezione definisce la strategia che è stata utilizzata per risolverlo in maniera efficiente. Nonostante la definizione del problema differisca dalle versioni più classiche intraviste negli approcci descritti all'interno dello stato dell'arte (si veda il capitolo 2), molti dei concetti precedentemente citati verranno implementati all'interno del seguente lavoro.

3.2.1 Intuizione

L'obiettivo alla base dell'algoritmo proposto è quello di riuscire a minimizzare quanto più possibile il numero di vertici distinti selezionati nel corso dell'intera esecuzione, non andando però a modificare la cardinalità della soluzione e il costo computazionale richiesti per ogni variazione della finestra temporale. Di conseguenza per riuscire a minimizzare il numero di vertici distinti selezionati durante l'intera esecuzione vi è la necessità di aggiornare l'Hitting Set in modo graduale tenendo traccia delle operazioni più importanti che avvengono al suo interno. La strategia si articola attraverso i seguenti pilastri:

- **Aggiornamento incrementale:** invece di ricalcolare l'Hitting Set da zero per ogni scorrimento della finestra temporale, la soluzione deve poter aggiornare la struttura esistente solo per gli intervalli di tempo inseriti.
- **Inserimento condizionato:** l'inserimento di nuovi nodi all'interno dell'Hitting Set deve dare la precedenza, in presenza di una condizione paritaria, ai nodi che sono già stati parte dell'Hitting Set di avere precedenza rispetto ad altri nodi esterni.
- **Ordinamento cronologico dell'Hitting Set:** tutti i nodi presenti all'interno dell'Hitting Set devono poter essere tracciati per permettere di essere scelti nuovamente in futuro.

Date le seguenti intuizioni generali di seguito riporto l'architettura di quella che in seguito è stata la soluzione da me proposta.

3.2.2 Architettura

Per descrivere l'architettura della soluzione proposta, l'intero ciclo di vita dell'algoritmo può essere ricondotto principalmente allo studio di una singola transizione temporale (o *time-slice*), che viene rappresentata nella figura 3.1.

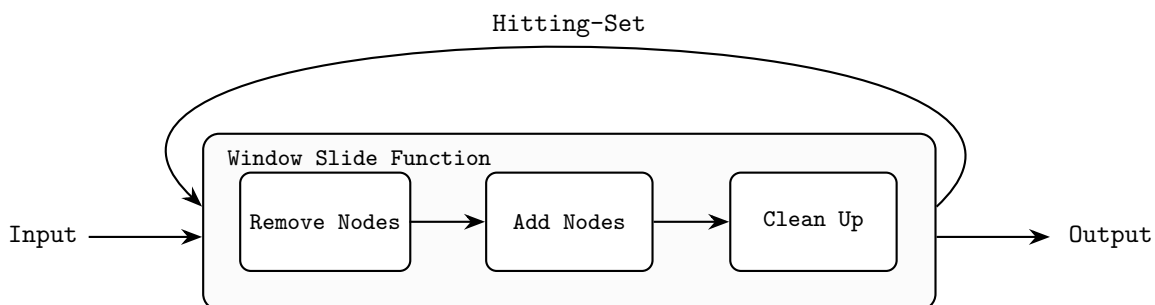


Figure 3.1: Architettura della funzione Sliding Window dell'algoritmo.

Tale immagine rappresenta un ciclo della **Window Slide Function**, ovvero la funzione che si occupa della gestione dell'Hitting Set durante lo shift della finestra temporale. Il comportamento del

framework può quindi essere modellato in modo ricorsivo. Assumendo che al tempo τ_k lo stato dell'Hitting Set sia noto e valido, il passaggio dal tempo τ_k al tempo τ_{k+1} attiva un flusso logico composto dalle seguenti macrooperazioni che verranno descritte nel dettaglio in seguito:

- **Remove Nodes:** operazione durante la quale avviene la rimozione degli iperarchi fuoriusciti dalla finestra temporale e dei nodi dell'Hitting Set non più fondamentali, vengono inoltre aggiornate strutture dati di supporto.
- **Add Nodes:** operazione durante la quale avviene l'aggiunta dei nuovi iperarchi e l'aggiunta dei nodi per completare l'Hitting Set già presente.
- **Clean Up:** operazione durante la quale avviene la rimozione dei nodi ridondanti presenti dopo l'unione del vecchio e del nuovo Hitting Set.

Composizione dell'Input

L'input che la funzione **Window Slide Function** riceve è diviso in due componenti principali. La prima, fornita dall'utente o dalla **Window Slide Function** precedente, è l'Hitting Set valido al tempo iniziale τ_k , indicata con $X(\tau_k)$, mentre la seconda, fornita dall'utente, è la lista degli iperarchi compresi all'interno della finestra temporale $[\tau_{k-1}, \tau_{k+1} + \epsilon]$, indicata con $W(\tau_k)$. Indicheremo in futuro l'insieme degli iperarchi compresi nell'intervallo temporale $[\tau_{k-1}, \tau_k]$, ovvero gli iperarchi in uscita come $W(\tau_k)^-$ e gli iperarchi compresi nell'intervallo temporale $[\tau_k + \epsilon, \tau_{k+1} + \epsilon]$, ovvero gli iperarchi in ingresso come $W(\tau_k)^+$.

Remove Nodes

La funzione di rimozione dei nodi prende in input la lista degli iperarchi in uscita dalla finestra temporale forniti dall'utente $W(\tau_k)^-$ e l'Hitting Set valido al tempo τ_k , indicato come $X(\tau_k)$. Una volta raccolti i dati forniti in input, la prima operazione da fare è quella di rimozione degli iperarchi presenti all'interno di $W(\tau_k)^-$ dalle strutture dati interne. Sia quindi

$$W'(\tau_k) = W(\tau_k) \setminus W(\tau_k)^-$$

l'insieme degli iperarchi che rimangono attivi dopo la fase di rimozione. Al termine della quale si effettua un controllo per verificare se i nodi precedentemente considerati all'interno dell'Hitting Set siano ancora validi o se non servano più a coprire degli archi altrimenti scoperti. La soluzione viene quindi ridotta ottenendo un insieme

$$X(\tau_k)^{needed} \subseteq X(\tau_k)$$

tale che:

$$\forall (S, t') \in W'(\tau_k), \quad X(\tau_k)^{needed} \cap S \neq \emptyset.$$

Al termine della modifica dell'Hitting Set si passa il risultato alla funzione **Add Nodes**.

Add Nodes

All'interno della funzione di aggiunta dei nodi si prendono come input gli iperarchi in ingresso $W(\tau_k)^+$, forniti dall'utente e l'Hitting Set valido dalla funzione di rimozione dei nodi, indicato con $X(\tau_k)^{needed}$. Dopodiché come primo passo, si effettua una scrematura degli iperarchi in ingresso eliminando quelli già coperti dalla soluzione corrente:

$$W(\tau_k)^{unc} = \left\{ (S, t') \in W(\tau_k)^+ \mid S \cap X(\tau_k)^{needed} = \emptyset \right\}.$$

L'insieme $W(\tau_k)^{unc}$ contiene dunque solamente gli iperarchi non coperti. Dopodiché si partiziona l'insieme $W(\tau_k)^{unc}$, in due sottoinsiemi disgiunti. Il primo contiene tutti gli iperarchi che contengono almeno un nodo che è stato selezionato almeno una volta all'interno del Hitting Set:

$$W(\tau_k)^{reuse} = \left\{ (S, t') \in W(\tau_k)^{unc} \mid S \cap R_{k-1} \neq \emptyset \right\},$$

dove

$$R_{k-1} = \bigcup_{i=0}^{k-1} X(i)$$

rappresenta l'insieme di tutti i vertici utilizzati almeno una volta fino al passo precedente. Il secondo invece contiene tutti gli iperarchi che non contengono nodi che sono stati selezionati all'interno dell'Hitting Set:

$$W(\tau_k)^{new} = \{(S, t') \in W(\tau_k)^{unc} \mid S \cap R_{k-1} = \emptyset\}.$$

Per costruzione risulta:

$$W(\tau_k)^{unc} = W(\tau_k)^{reuse} \cup W(\tau_k)^{new}, \quad W(\tau_k)^{reuse} \cap W(\tau_k)^{new} = \emptyset.$$

Successivamente si effettua una ricerca del Minimum Hitting Set (MHS) attraverso l'utilizzo di metodologie greedy [15, 7] sull'insieme $W(\tau_k)^{new}$, creando così l'insieme $X(\tau_k)^{new}$. Infine si rimuovono da $W(\tau_k)^{reuse}$ tutti gli iperarchi contenenti almeno un elemento del nuovo Hitting Set $X(\tau_k)^{new}$:

$$W(\tau_k)^{remaining} = \{(S, t') \in W(\tau_k)^{reuse} \mid S \cap X(\tau_k)^{new} = \emptyset\}$$

e sugli iperarchi rimanenti si effettua un'ultima ricerca del Minimum Hitting Set (MHS) attraverso l'utilizzo di metodologie greedy sull'insieme $W(\tau_k)^{remaining}$ dando però priorità alla scelta dei nodi appartenenti a R_{k-1} per garantire il riutilizzo massimale dei nodi. Quest'ultima operazione crea l'insieme $X(\tau_k)^{reuse}$, ovvero l'Hitting Set degli iperarchi con nodi riutilizzabili.

Clean Up

La fase finale di **Clean Up** consiste nel prendere gli Hitting Set risultanti dalle precedenti operazioni, dunque: $X(\tau_k)^{needed}$, $X(\tau_k)^{new}$ e $X(\tau_k)^{reuse}$ e unirli in un solo Hitting Set generale attraverso la seguente operazione:

$$X(\tau_{k+1})^{complete} = X(\tau_k)^{needed} \cup X(\tau_k)^{new} \cup X(\tau_k)^{reuse}.$$

Successivamente si esegue un controllo per verificare se ciascun nodo presente all'interno dell'Hitting Set $X(\tau_{k+1})^{complete}$ copra da solo almeno un iperarco e si rimuovono tutti quelli che non rispettano tale regola

$$X(\tau_{k+1})^{reduced} = \left\{ X(\tau_{k+1})^{complete} \setminus X(\tau_{k+1})^{redundant} \right\}$$

dove

$$X(\tau_{k+1})^{redundant} = \left\{ v \in X(\tau_{k+1})^{complete}, \nexists (S, t') \in W' \mid S \cap X(\tau_{k+1})^{complete} = \{v\} \right\}.$$

Infine si controlla se il nuovo Hitting Set $X(\tau_{k+1})^{reduced}$ copre completamente gli iperarchi attivi e nel caso in cui la copertura non fosse globale si applica un'ultima volta l'algoritmo greedy con priorità per la ricerca del Minimum Hitting Set (MHS) sugli elementi W' che non sono ancora stati coperti, $X(\tau_{k+1})^{patch}$. Alla fine di tutto si determina quindi

$$X(\tau_{k+1}) = X(\tau_{k+1})^{reduced} \cup X(\tau_{k+1})^{patch}.$$

Composizione dell'Output

La funzione **Window Slide Function** rilascia infine in output l'insieme $X(\tau_{k+1})$, ovvero il Minimum Hitting Set (MHS) calcolato per la finestra temporale che si estende da τ_{k+1} a $\tau_{k+1} + \epsilon$.

3.3 Descrizione dell'algoritmo

Una volta delineata l'architettura concettuale e la strategia incrementale nella sezione precedente, questo capitolo scende nel dettaglio ingegneristico e implementativo della soluzione. Di seguito verranno presentate le strutture dati fondamentali scelte per garantire l'efficienza computazionale, lo pseudocodice dell'algoritmo e un'analisi passo-passo del suo funzionamento.

3.3.1 Strutture dati utilizzate

Per garantire alle operazioni di aggiornamento incrementali un costo computazionale contenuto, è stato necessario l'utilizzo di strutture dati complesse che ci permettessero di ammortizzare e diminuire i costi computazionali del problema. Per le strutture dati è stato preso spunto da quelle utilizzate per l'analisi del problema del Minimum Hitting Set sviluppato in ambito dinamico (si veda il Capitolo 2.3) [1]. Le strutture dati principali che sono state utilizzate all'interno del mio lavoro sono le seguenti:

- **Ipergrafo Temporale:** Rappresentato tramite la struttura dati TemporalHypergraph fornita dalla libreria “hypergraphx” [18], serve a mantenere l'insieme degli iperarchi e dei rispettivi timestamp, ovvero gli istanti di tempo che determinano l'arrivo degli iperarchi all'interno della finestra temporale.
- **Hitting Set:** Rappresentato tramite un set, serve a tenere traccia dell'Hitting Set attivo ad un generico tempo τ_k .
- **Iperarchi coperti singolarmente:** Rappresentato tramite un dizionario, serve a tenere traccia del numero di iperarchi coperti singolarmente da parte di ogni nodo presente nell'Hitting Set.
- **Presenza dei nodi nell'Hitting Set:** Rappresentato tramite un dizionario, serve a tenere traccia dell'ultimo istante temporale all'interno del quale un dato nodo $v \in V$ è stato parte dell'Hitting Set.

3.3.2 Lo Pseudocodice iniziale

Di seguito viene riportato lo pseudocodice della Window Slide Function che descrive la logica di transizione incrementale dall'istante τ_k all'istante τ_{k+1} .

Algorithm 1 Window Slide Function

Require: $X(\tau_k)$, $W(\tau_k)$, $CoverCount(\tau_k)$, $NodePresence(\tau_k)$

Ensure: $W(\tau_{k+1})$, $CoverCount(\tau_{k+1})$, $NodePresence(\tau_{k+1})$

- 1: $X(\tau_k)^{needed} \leftarrow RemoveNodes(X(\tau_k), W(\tau_k)^-, CoverCount(\tau_k))$
 - 2: $(W(\tau_k)^{new}, W(\tau_k)^{reuse}) \leftarrow CleanupAndArchDivision(W(\tau_k)^+, X(\tau_k)^{needed}, NodePresence(\tau_k))$
 - 3: $X(\tau_k)^{new} \leftarrow GreedyHittingSet(W(\tau_k)^{new})$
 - 4: $W(\tau_k)^{remaining} \leftarrow \{(S, t') \in W(\tau_k)^{reuse} \mid S \cap X(\tau_k)^{new} = \emptyset\}$
 - 5: $X(\tau_k)^{reuse} \leftarrow ReuseGreedyHittingSet(W(\tau_k)^{remaining}, NodePresence(\tau_k))$
 - 6: $X(\tau_{k+1})^{complete} \leftarrow X(\tau_k)^{needed} \cup X(\tau_k)^{new} \cup X(\tau_k)^{reuse}$
 - 7: $X(\tau_{k+1}) \leftarrow Cleanup(X(\tau_{k+1})^{complete}, W^+(\tau_k), CoverCount(\tau_k), NodePresence(\tau_k))$
 - 8: $NodePresence(\tau_{k+1}) \leftarrow UpdateNodePresence(NodePresence(\tau_k), X(\tau_{k+1}))$
 - 9: **return** $X(\tau_{k+1})$, $CoverCount(\tau_{k+1})$, $NodePresence(\tau_{k+1})$
-

Poiché la funzione sfrutta la manipolazione degli iperarchi e il riuso dei nodi è necessario analizzare anche le sottofunzioni che la compongono per comprendere al meglio il funzionamento dell'algoritmo:

- **RemoveNodes:** la seguente funzione come descritto in precedenza ha lo scopo di rimuovere tutti i nodi dell'Hitting Set che successivamente alla rimozione degli iperarchi non coprono più effettivamente alcun iperarco da soli.
- **CleanupAndArchDivision:** all'interno della funzione **CleanupAndArchDivision** si controlla per ogni iperarco se è presente o meno un nodo all'interno dell'insieme $X(\tau_k)^{needed}$, se è presente si scarta l'iperarco poiché già coperto, altrimenti lo si aggiunge all'insieme $W(\tau_k)^{reuse}$ o $W(\tau_k)^{new}$ in base a se possiede nodi presenti in $NodePresence(\tau_k)$ oppure no.
- **GreedyHittingSet:** l'algoritmo utilizzato per ricavare l'Hitting Set dato l'ipergrafo di partenza è indifferente e può essere trattato come una “scatola nera” (o *Black-Box*), in quanto il framework è agnostico rispetto alla versione di variante Greedy che viene scelta, poiché l'algoritmo viene fatto girare sugli iperarchi che non hanno nodi presenti all'interno di $NodePresence(\tau_k)$, l'unico nostro obiettivo è quello di coprire questi iperarchi con il minor numero di nodi possibili.

- **ReuseGreedyHittingSet**: come discusso in precedenza per la funzione **GreedyHittingSet** anche in questo caso è possibile adottare un qualunque algoritmo greedy in grado di risolvere il problema del Minimum Hitting Set, con l'unica variazione di rispettare un'ulteriore euristica di priorità (**NodePresence**) forzando l'algoritmo in caso di indecisione a preferire nodi con uno storico in modo da ridurre l'aggiunta di nuovi nodi distinti all'interno dell'Hitting Set.
- **Cleanup**: la funzione di **Cleanup** ha lo scopo di rimuovere tutti i nodi ridondanti, ovvero quelli che non coprono da soli nessun iperarco e di testare se $X(\tau_{k+1})^{reduced}$ riesce a coprire tutti gli iperarchi presenti all'interno della finestra temporale, in caso contrario si utilizza la funzione **ReuseGreedyHittingSet** sugli iperarchi non coperti da $X(\tau_{k+1})^{reduced}$ andando così a ricavare l'insieme $X(\tau_{k+1})^{patch}$ in modo da creare $X(\tau_{k+1})$.
- **UpdateNodePresence**: la seguente funzione ha il solo scopo di aggiornare il counter di presenza all'interno dell'Hitting Set per tutti i nodi alla fine dello shift temporale.

3.3.3 Analisi Passo-Passo

L'Algoritmo **Window Slide Function 1** serve a definire le operazioni necessarie da effettuare successivamente allo shift della finestra temporale per garantire la validità dell'Hitting Set. Di seguito viene presentata un'analisi dettagliata del flusso di esecuzione, per evidenziare come le sottofunzioni precedentemente descritte cooperino per mantenere la minimalità dell'intera soluzione.

Riga 1, la funzione **RemoveNodes** riceve in input l'Hitting Set valido al tempo τ_k $X(\tau_k)$, l'insieme degli iperarchi usciti dalla finestra temporale a seguito dello shift $W(\tau_k)^-$ e la struttura dati che tiene traccia di quanti iperarchi vengono coperti singolarmente da ogni nodo presente in $X(\tau_k)$. Per garantire il mantenimento del minor numero di nodi andiamo ad eliminare tutti quelli che non coprono singolarmente neanche un iperarco, generando l'insieme di nodi strettamente necessari denominato $X(\tau_k)^{needed}$.

Riga 2, il framework affronta l'inserimento dei nuovi iperarchi ($W(\tau_k)^+$) tramite la funzione **CleanUp-AndArchDivision**. Questa funzione esegue un duplice compito: da un lato scarta i nuovi iperarchi che risultano già coperti dai nodi dell'Hitting Set rimanente $X(\tau_k)^{needed}$; mentre dall'altro, suddivide i restanti iperarchi scoperti in due liste ($W(\tau_k)^{new}$ e $W(\tau_k)^{reuse}$), ovvero rispettivamente la lista degli iperarchi con almeno un nodo che è stato precedentemente all'interno di un Hitting Set e il suo corrispettivo. Tale operazione si basa sulla struttura dati **NodePresence**(τ_k), isolando gli iperarchi che contengono nodi già visti storicamente dal sistema.

Le **righe 3, 4 e 5** effettuano l'operazione di **AddNode** precedentemente discussa nell'architettura (si veda il Capitolo 3.2). Alla riga 3, viene invocato il **GreedyHittingSet** standard sull'insieme $W(\tau_k)^{new}$ per coprire gli iperarchi senza nodi storicamente già visti utilizzando il numero minore possibile di nodi, producendo l'insieme $X(\tau_k)^{new}$. Successivamente alla riga 4, si rimuovono dalla lista $W(\tau_k)^{reuse}$ tutti gli iperarchi che sono stati coperti dall'introduzione dei nodi in $X(\tau_k)^{new}$. Su quest'ultimo insieme viene eseguito alla riga 5 la funzione **ReuseGreedyHittingSet** che, sfruttando i pesi di **NodePresence**(τ_k), cerca di trovare il numero minore di nodi che coprono gli iperarchi presenti in $W(\tau_k)^{remaining}$ massimizzando allo stesso tempo il riuso dei vertici storici e generando così l'insieme $X(\tau_k)^{reuse}$.

Le **righe 6, 7 e 8** controllano la soluzione e consolidano lo stato delle strutture dati. Dopo aver unito i tre contributi dei nodi alla riga 6, viene chiamata a riga 7 la funzione **Cleanup** la quale effettua un controllo di sicurezza per eliminare eventuali ridondanze incrociate nate dalle due diverse fasi greedy e, se necessario, applica una "patch" locale per garantire la totale copertura della nuova finestra temporale $W'(\tau_k)$. Il ciclo si conclude alla riga 8 con l'aggiornamento dei contatori in **NodePresence**(τ_{k+1}), preparando le strutture dati per lo shift successivo.

Riga 9, viene infine ritornato dalla funzione l'Hitting Set valido al tempo τ_{k+1} e vengono restituite le strutture dati aggiornate.

3.3.4 Esempio di Esecuzione

Per rendere concreto il funzionamento algoritmico della *Window Slide Function*, se ne illustra l'applicazione su un caso di test isolato durante la transizione dall'istante τ_k all'istante τ_{k+1} , una rappresentazione

grafica degli archi la si può riscontrare nella fig 3.2. Assumiamo che al tempo iniziale τ_k si verifichino

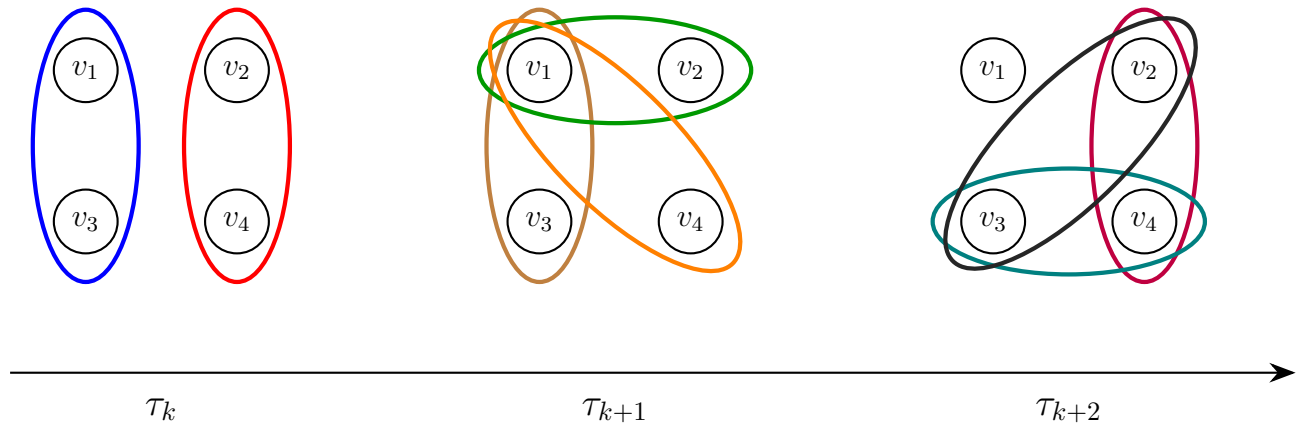


Figure 3.2: Evoluzione temporale dell'ipergrafo nei tre istanti τ_k , τ_{k+1} e τ_{k+2} .

le seguenti condizioni:

- Finestra temporale = $[\tau_k, \tau_{k+1}]$
- $X(\tau_k) = \{v_1, v_2\}$ (Hitting Set corrente).
- $CoverCount(\tau_k)$ indica che v_1 copre singolarmente 2 iperarchi, mentre v_2 copre singolarmente 1 iperarco.
- $NodePresence(\tau_k) = \{v_1 : k, v_2 : k, v_3 : 0, v_4 : 0\}$, indicando la frequenza storica dei nodi.

Al tempo τ_{k+1} , l'evoluzione temporale comporta le seguenti modifiche strutturali:

- Gli iperarchi $e_{blu} = \{v_1, v_3\}$ e $e_{rosso} = \{v_2, v_4\}$ scadono ed entra in $W(\tau_k)^-$.
- Entrano tre nuovi iperarchi in $W(\tau_k)^+$: $e_{grigio} = \{v_2, v_3\}$, $e_{viola} = \{v_2, v_4\}$ e $e_{azzurro} = \{v_3, v_4\}$.

L'algoritmo esegue i passi come segue:

1. **Rimozione nodi obsoleti (Riga 1):** Viene processata la scadenza di e_{blu} e e_{rosso} . Poiché v_2 non copre nessun altro iperarco attivo, la funzione **RemoveNodes** lo elimina. Il nodo v_1 viene invece mantenuto poiché copre ancora e_{verde} , e_{giallo} e $e_{marrone}$. Si ottiene $X(\tau_k)^{needed} = \{v_1\}$.
2. **Divisione degli archi (Riga 2):** Viene esaminato l'insieme dei nuovi arrivi $W(\tau_k)^+ = \{e_{grigio}, e_{viola}, e_{azzurro}\}$.
 - L'iperarco $e_{grigio} = \{v_2, v_3\}$ non è coperto da $X(\tau_k)^{needed}$. Controllando $NodePresence$, si nota che il nodo v_2 ha uno storico (k), mentre v_3 è un nodo vergine (0). Di conseguenza, e_{grigio} viene inserito in $W(\tau_k)^{reuse}$. L'insieme $W(\tau_k)^{new}$ rimane vuoto.
 - L'iperarco $e_{viola} = \{v_2, v_4\}$ non è coperto da $X(\tau_k)^{needed}$. Controllando $NodePresence$, si nota che il nodo v_2 ha uno storico (k), mentre v_4 è un nodo vergine (0). Di conseguenza, e_{viola} viene inserito in $W(\tau_k)^{reuse}$. L'insieme $W(\tau_k)^{new}$ rimane vuoto.
 - L'iperarco $e_{azzurro} = \{v_3, v_4\}$ non è coperto da $X(\tau_k)^{needed}$. Controllando $NodePresence$, si nota che sia il nodo v_3 che v_4 sono nodi vergine (0). Di conseguenza, $e_{azzurro}$ viene inserito in $W(\tau_k)^{new}$.
3. **Fase Greedy (Righe 3-5):** Viene applicata la funzione **GreedyHittingSet** sull'insieme $W(\tau_k)^{new}$, portando $(X(\tau_k)^{new} = \{v_4\})$. Alla riga 4, si rimuove l'iperarco e_{viola} da $W(\tau_k)^{reuse}$ in quanto è ora coperto da uno dei nodi presenti in $X(\tau_k)^{new}$. Alla riga 5, la funzione **ReuseGreedyHittingSet** viene applicata sull'insieme $W(\tau_k)^{remaining} = \{e_{grigio}\}$. L'algoritmo valuta i due nodi e grazie al peso maggiore in $NodePresence$, preferisce lo storico v_2 rispetto al nuovo v_3 . Viene determinato $X(\tau_k)^{reuse} = \{v_2\}$.

4. **Unione e Pulizia (Righe 6-8):** L'unione temporanea genera $X(\tau_{k+1})^{complete} = \{v_1\} \cup \{v_2\} \cup \{v_4\} = \{v_1, v_2, v_4\}$. La funzione **Cleanup** convalida l'insieme verificando che sia minimale e che copra l'intera finestra aggiornata. Infine, **UpdateNodePresence** incrementa i contatori di v_1 , v_2 e v_4 .

L'algoritmo restituisce quindi la soluzione aggiornata $X(\tau_{k+1}) = \{v_1, v_2, v_4\}$.

3.4 Considerazioni sulla complessità

In questa sezione viene proposta un'analisi della complessità computazionale, temporale e spaziale, dell'algoritmo *Window Slide Function* (Algoritmo 1). L'obiettivo è quello di dimostrare come l'approccio incrementale permetta di ridurre i costi computazionali dell'algoritmo in quanto non costringe a un ricalcolo della soluzione da zero ad ogni spostamento della finestra temporale.

Analisi Temporale

Per riuscire a determinare la complessità temporale complessiva, è necessario esaminare dapprima il costo computazionale delle singole procedure ausiliarie:

- **RemoveNodes:** Questa funzione analizza gli m^- iperarchi scaduti. Sfruttando la struttura *CoverCount*, l'algoritmo decrementa i contatori di copertura in tempo $O(1)$ per ciascun nodo appartenente all'iperarco scaduto. Poiché ogni iperarco ha una cardinalità massima pari a k , il costo totale è limitato superiormente da $O(m^- \cdot k)$.
- **CleanUpAndArchDivision:** Per ognuno degli m^+ nuovi iperarchi arrivati, la funzione controlla l'intersezione con l'insieme $X(\tau_k)^{needed}$ di dimensione massima n . Considerando l'inserimento degli iperarchi in lista come costo $O(1)$ ammortizzato e considerando che nel caso peggiore per ogni iperarco andranno analizzati tutti i nodi, si ha un costo computazionale di $O(m^+ \cdot k)$ per controllare tutti gli iperarchi e un costo di $O(1)$ per l'inserimento degli iperarchi nelle rispettive liste. Di conseguenza, la funzione è limitata superiormente da $O(m^+ \cdot k)$.
- **GreedyHittingSet e ReuseGreedyHittingSet:** Sia m_{res} il numero di iperarchi rimasti scoperti che devono essere processati dalle fasi greedy (dove $m_{res} \leq m^+$). Un algoritmo greedy classico per l'Hitting Set ha una complessità pari a $O(m_{res} \cdot k \cdot \log n)$ o $O(m_{res} \cdot k^2)$ a seconda dell'implementazione utilizzata [15, 7]. Nel nostro framework, l'aggiunta dell'euristica basata su *NodePresence* nel **ReuseGreedyHittingSet** richiede una consultazione in tempo costante $O(1)$ tramite hash map, mantenendo la complessità asintotica invariata rispetto al greedy tradizionale.
- **Cleanup:** La fase di potatura verifica la minimalità dell'Hitting Set finale eseguendo un controllo su tutti gli iperarchi presenti all'interno della finestra temporale aggiornata (m') e, nel caso in cui fosse necessario, eseguendo nuovamente la funzione **ReuseGreedyHittingSet**. Poiché, come visto in precedenza, il costo della visita di ciascun iperarco è $O(m' \cdot k)$ e il costo computazionale della funzione **ReuseGreedyHittingSet** è di $O(m' \cdot k \cdot \log n)$, il costo computazionale totale della funzione **Cleanup** è $O(m' \cdot k + m' \cdot k \cdot \log n)$.

Sommando i contributi delle singole fasi, la complessità temporale della *Window Slide Function* nel caso peggiore è dominata dalla fase di calcolo euristico effettuata durante il clean up finale, portando il costo computazionale a essere:

$$O(m^- \cdot k + m' \cdot k \cdot \log n)$$

Il vero vantaggio computazionale risiede però nel fatto che, in uno scenario reale, all'interno del quale le variazioni della finestra sono ridotte ($m^-, m^+ \ll m'$) e la funzione greedy del clean up non deve effettuare un ricalcolo completo di tutto l'ipergrafo, il costo computazionale medio è inferiore rispetto a quello proposto nel caso pessimo teorico.

Complessità Spaziale

La complessità spaziale viene determinata in base alle strutture dati necessarie a preservare lo stato tra uno scorrimento della finestra temporale e l'altro:

- La rappresentazione della finestra corrente occupa uno spazio pari a $O(m' \cdot k)$.
- Le strutture di tracciamento *CoverCount* e *NodePresence* richiedono uno spazio lineare rispetto al numero di nodi totali del sistema, ovvero nel caso peggiore $O(n)$.

La complessità spaziale complessiva è quindi $O(m' \cdot k + n)$.

4 Valutazioni sperimentali

Conclusa la fase di definizione algoritmica e di modellazione teorica della soluzione, questo capitolo si concentra sull'analisi sperimentale dell'approccio proposto. L'obiettivo di tale analisi è sia quello di verificare l'efficienza computazionale del sistema in termini di tempi di risposta e scalabilità; sia quello di valutare l'efficacia e la robustezza dell'algoritmo nella ricerca di soluzioni di alta qualità al variare della complessità del problema.

4.1 Configurazione e ambiente di test

Gli esperimenti che seguiranno all'interno di questo capitolo sono stati condotti all'interno dello stesso ambiente e sulla stessa macchina, in modo da poter garantire omogeneità tra i vari casi di test. Inoltre, per garantire confrontabilità e ripetitività dei test eseguiti procedo a specificare le caratteristiche tecniche delle componenti da me utilizzate.

- **Hardware:** i test sono stati effettuati nella loro interezza su un laptop con processore AMD Ryzen 7 5700U e con 8 GB di memoria RAM. Poiché l'algoritmo proposto non implementa logiche di parallelizzazione, le prestazioni riportate si riferiscono esclusivamente all'esecuzione del processo in modalità *single-thread*.
- **Software:** il programma è stato sviluppato interamente in Python (v 3.14), sfruttando delle librerie di supporto quali `hypergraphx` [18] per la gestione degli ipergrafi temporali e `time` per la misurazione dei tempi di esecuzione.

4.2 Varianti degli algoritmi analizzati

Per analizzare l'impatto che ogni componente della soluzione implementata ha sul sistema intero abbiamo deciso di testare diverse versioni dello stesso algoritmo, variando per ognuna di esse una scelta progettuale, in modo da valutarne l'importanza relativa. Questo approccio permette di isolare il contributo di una scelta progettuale alla volta, evitando che le differenze osservate tra le configurazioni siano attribuibili a più fattori sovrapposti. Sono state definite quattro varianti diverse di algoritmi, riassunte di seguito:

- **Iterative Greedy:** questo algoritmo consiste in una versione nella quale viene applicato un algoritmo greedy in maniera naïve, che ricalcola da capo l'intero Hitting Set ad ogni spostamento della finestra temporale.
- **Sliding different Greedy:** questo algoritmo rappresenta una variazione di quello presentato nella soluzione proposta (si veda il Capitolo 3.2); in particolare vengono modificate le funzioni `GreedyHittingSet` e `ReuseGreedyHittingSet`, applicando una tipologia diversa di algoritmo greedy per valutare il contributo di tale algoritmo sia dal punto di vista del tempo di esecuzione che della dimensione media dell'Hitting Set risultante.
- **Sliding New Nodes:** questo algoritmo rappresenta una variazione di quello presentato nella soluzione proposta (si veda il Capitolo 3.2); in particolare viene modificata la gestione dei pesi all'interno della funzione `ReuseGreedyHittingSet`, i pesi vengono invertiti portando l'algoritmo a preferire la scelta di nodi senza uno storico all'interno degli Hitting Set precedenti.
- **Sliding Reuse:** questo algoritmo coincide con quello presentato nella soluzione proposta (si veda il Capitolo 3.2).

Algoritmi greedy utilizzati

Nel corso della descrizione della soluzione proposta (si veda il Capitolo 3.2) e della precedente esposizione delle varianti algoritmiche utilizzate nella fase di test, sono stati citati più volte alcuni algoritmi risolutivi greedy, senza però aver approfondito i rispettivi dettagli implementativi. Come discusso nella sezione delle soluzioni euristiche per il problema dell'Hitting Set analizzato in ambito statico (si veda la Sezione 2.2), gli algoritmi greedy per il Minimum Hitting Set possono essere classificati in euristiche *vertex-oriented*, che privilegiano il contributo globale alla copertura, ed euristiche *hyperedge-oriented*, che operano invece una scelta locale basata sull'iperarco analizzato [10]. Gli algoritmi greedy utilizzati all'interno di questo lavoro fanno parte della famiglia delle euristiche *hyperedge-oriented*, più precisamente sono state utilizzare due varianti distinte:

- **Nodo di cardinalità massima in iperarco minimo:** variante utilizzata nelle versioni **Iterative Greedy**, **Sliding New Nodes** e **Sliding Reuse**. Consiste nella pratica di selezionare come nuovo nodo dell'Hitting Set il nodo di cardinalità massima che fa parte dell'iperarco non coperto di cardinalità minima. La versione della funzione **ReuseGreedyHittingSet** si differenzia dalla funzione **GreedyHittingSet** in quanto in caso di più nodi con la stessa cardinalità massima si seleziona quello con uno storico all'interno degli Hitting Set precedenti, eccezione fatta per la versione **Sliding New Nodes** che come spiegato in precedenza inverte la scelta di priorità.
- **Nodo di cardinalità massima in iperarco randomico:** variante utilizzata nella versione **Sliding different Greedy**. Consiste nella pratica di selezionare come nuovo nodo dell'Hitting Set il nodo di cardinalità massima che fa parte di un iperarco non coperto selezionato in maniera randomica. La versione della funzione **ReuseGreedyHittingSet** si differenzia dalla funzione **GreedyHittingSet** in quanto in caso di più nodi con la stessa cardinalità massima si seleziona quello con uno storico all'interno degli Hitting Set precedenti.

4.3 Istanze di benchmark e caratterizzazione dei dati

Per valutare le prestazioni degli algoritmi presentati nel capitolo precedente è necessario specificare anche l'insieme dei dataset sui quali sono stati fatti girare gli algoritmi. Per valutare al meglio la robustezza e la capacità di generalizzazione degli algoritmi proposti, i test sono stati condotti su un insieme di quattro istanze diversificate. Per generalizzare il più possibile sono stati presi in considerazione dataset differenti in dimensione e distribuzione temporale, l'unico elemento comune è che tutte le istanze sono state tratte da dati reali ampiamente utilizzati nella letteratura degli ipergrafi temporali [18]. Le caratteristiche strutturali ed evolutive delle istanze selezionate sono riassunte nella seguente Tabella 4.1.

Table 4.1: Proprietà strutturali delle istanze di benchmark utilizzate.

ID Dataset	Nodi ($ V $)	Iperarchi ($ E $)	Iperarchi unici ($ E $)
Contacts high school	$3 \cdot 10^3$	$1.7 \cdot 10^5$	$7.8 \cdot 10^3$
Coauth history	$1 \cdot 10^6$	$1.8 \cdot 10^6$	$9 \cdot 10^5$
Email Enron	$8.4 \cdot 10^4$	$2.4 \cdot 10^5$	$1.1 \cdot 10^5$
Crypto bitcoin	$2.5 \cdot 10^6$	$1.5 \cdot 10^6$	$1.3 \cdot 10^6$

Analisi delle distribuzioni strutturali

Un fattore cruciale per determinare l'efficacia della soluzione basata sulla finestra temporale a scorrimento è la distribuzione intrinseca dei dati. La seguente figura (si veda Figura 4.5) mostra la distribuzione che definisce il numero di iperarchi che arrivano al tempo τ_k per le istanze principali. Come si può notare dai profili temporali riportati in figura, i dataset analizzano dinamiche evolutive profondamente differenti tra di loro. Alcuni di essi sono caratterizzati da periodi di *burst*, ovvero da momenti di picco di attività e da momenti di assenza di iperarchi, mentre altre hanno una distribuzione caratterizzata da un picco di attività ben definito. Questa forte asimmetria nella distribuzione gioca un ruolo fondamentale nei risultati che ci si può aspettare dall'algoritmo sviluppato.

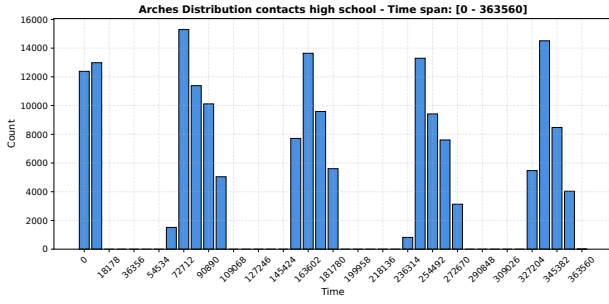


Figure 4.1: *

(a) Dataset Contacts high school

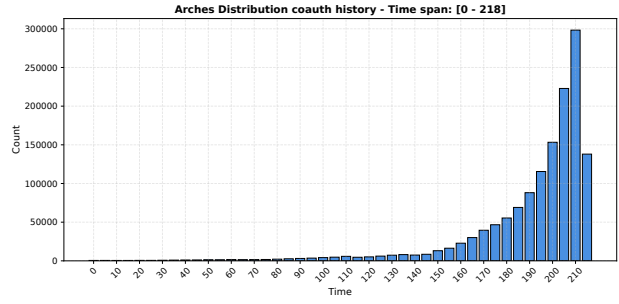


Figure 4.2: *

(b) Dataset Coauth history

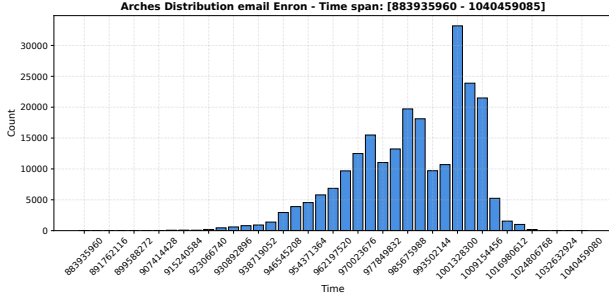


Figure 4.3: *

(c) Dataset Email Enron

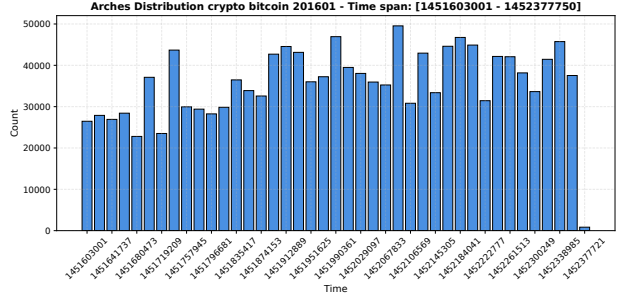


Figure 4.4: *

(d) Dataset Crypto bitcoin

Figure 4.5: Distribuzione delle frequenze di interazioni nei dataset analizzati.

4.4 Analisi delle prestazioni computazionali

In questa sezione vengono presentati e analizzati i riscontri empirici ottenuti dall'esecuzione della soluzione proposta (si veda il Capitolo 3.2) mettendoli a confronto con delle versioni di confronto (si veda il Capitolo 4.2) sulle istanze di benchmark precedentemente caratterizzate (si veda il Capitolo 4.3). L'obiettivo fondamentale è quello di quantificare l'efficienza e l'efficacia dell'approccio basato sulla finestra temporale a scorrimento, evidenziandone la scalabilità al crescere della dimensionalità del problema.

4.4.1 Metriche di valutazione e protocollo di tesi

Per valutare qualitativamente il comportamento del sistema sono state analizzate le tre metriche precedentemente citate durante la definizione formale del problema (si veda il Capitolo 3.1), ovvero:

- **Tempo di esecuzione (T_{exec}):** espresso in secondi, misura il tempo impiegato da parte dell'elaboratore per arrivare al termine dell'esecuzione dell'algoritmo analizzato.
- **Dimensione media dell'Hitting Set ($\overline{|X|}_\tau$):** espresso attraverso il numero di nodi, misura la cardinalità media temporale dell'insieme di copertura calcolato durante l'intero periodo di esecuzione.
- **Numero nodi distinti scelti (R):** espresso attraverso il numero di nodi, rappresenta il volume complessivo di vertici unici accumulati nell'Hitting Set globale al termine del processo (si veda il Capitolo 3.1).

L'analisi congiunta delle tre metriche permette di mappare accuratamente il trade-off dell'algoritmo proposto tra l'efficienza computazionale del codice e la qualità delle soluzioni generate, offrendo una panoramica completa sulla scalabilità dell'approccio proposto.

4.4.2 Aspettative sperimentali e risultati

Prima di analizzare direttamente i risultati sperimentali, è importante definire un comportamento atteso dell'algoritmo in tutte le sue versioni in modo da poter successivamente confrontare le aspettative con i risultati effettivi. Poiché l'efficienza della soluzione si basa parzialmente anche sul rapporto tra la dimensione della finestra temporale considerata (ϵ) e la dimensione dello scorrimento computazionale

della stessa (δ), abbiamo deciso, per ogni dataset analizzato, di testare tre rapporti differenti tra ϵ e δ in modo da poter analizzare come cambia il risultato al variare delle loro dimensioni.

Contacts high school

La caratteristica particolare di questo dataset è la distribuzione di arrivo degli iperarchi marcatamente intermittente (si veda Figura 4.5 a). Su tale base si può affermare che l'efficacia dell'algoritmo proposto e delle rispettive estensioni può essere garantita solo in base al rapporto che la dimensione della finestra temporale e la dimensione del relativo scorrimento hanno con i picchi e le valli di attività della distribuzione mostrata. La soluzione proposta riesce dunque a garantire prestazioni ottimali solo a condizione che l'ampiezza della finestra temporale sia sufficiente a intercettare più picchi concorrenti e che il passo di scorrimento sia almeno pari al loro tempo di inter-arrivo. In pratica l'unico modo affinché l'algoritmo riesca a funzionare con questa distribuzione degli archi in ingresso è selezionando delle dimensioni di ϵ e δ in grado di mascherare la presenza dei burst temporali.

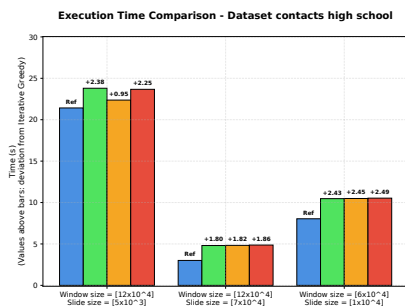


Figure 4.6: *
Tempo di esecuzione

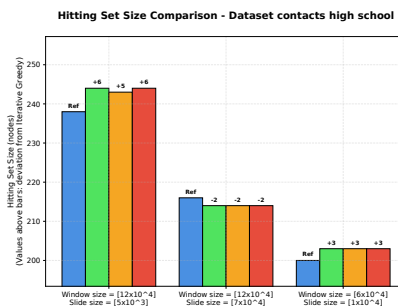


Figure 4.7: *
Dimensione media Hitting Set

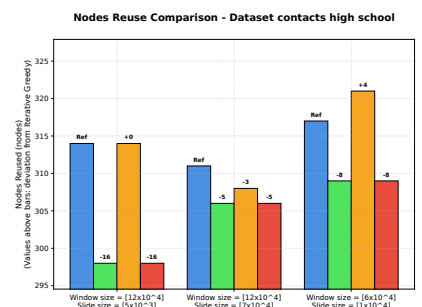


Figure 4.8: *
Numero nodi distinti scelti

— Iterative Greedy — Sliding different Greedy — Sliding New Nodes — Sliding Reuse

Coauth history

Come nel grafico precedente, la caratteristica particolare del dataset risiede nella sua distribuzione di arrivo degli iperarchi (si veda Figura 4.5 b). La presenza di un picco di nodi verso la fine del periodo temporale studiato non permette di sfruttare la conoscenza acquisita dalle finestre precedenti, limitando le situazioni in cui l'algoritmo riesce a fornire prestazioni migliori.

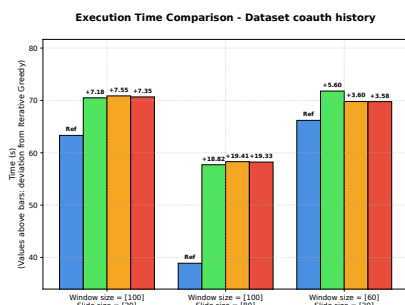


Figure 4.9: *
Tempo di esecuzione

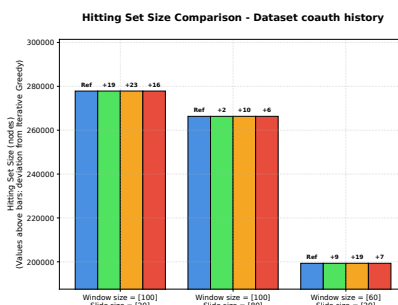


Figure 4.10: *
Dimensione media Hitting Set

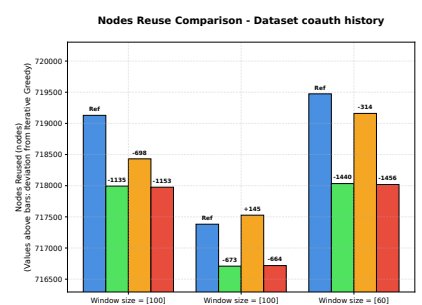


Figure 4.11: *
Numero nodi distinti scelti

— Iterative Greedy — Sliding different Greedy — Sliding New Nodes — Sliding Reuse

Email Enron

A differenza dei grafici precedenti, la distribuzione di arrivo degli iperarchi del dataset non sembra poter creare particolari problemi, in quanto la fase di picco è situata a metà del periodo temporale studiato (si veda Figura 4.5 c). Tale dataset però risulta essere particolarmente denso (si veda Tabella 4.1), fattore che potrebbe portare a una minore differenza nel numero di nodi distinti scelti (R) e a un buon grado di approssimazione dell'Hitting Set sia da parte degli algoritmi basati su finestra temporale a scorrimento, sia da parte di quelli greedy iterativi.

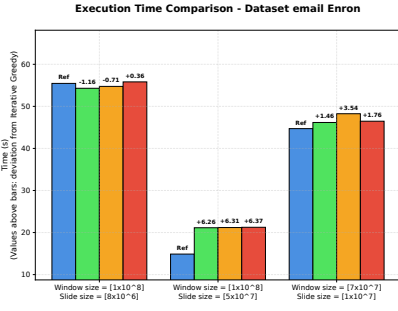


Figure 4.12: *
Tempo di esecuzione

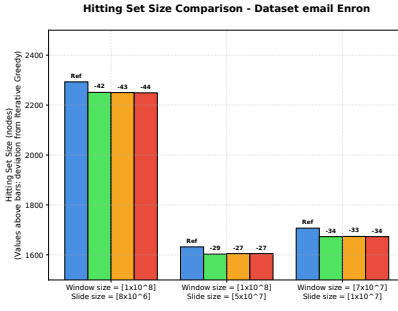


Figure 4.13: *
Dimensione media Hitting Set

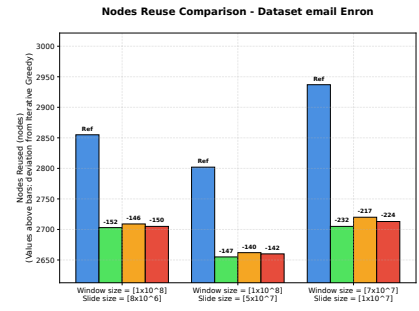


Figure 4.14: *
Numero nodi distinti scelti

— Iterative Greedy — Sliding different Greedy — Sliding New Nodes — Sliding Reuse

Crypto bitcoin

L'ultimo dataset risulta essere quello più favorevole dal punto di vista dell'algoritmo da me sviluppato, in quanto presenta una distribuzione di arrivo quasi costante e di una struttura non particolarmente densa. In presenza di tali condizioni, ci aspettiamo che il nostro algoritmo riesca a sfruttare al meglio il funzionamento della finestra temporale, mentre, a causa della natura sparsa del dataset, ci aspettiamo che gli algoritmi greedy iterativi non riescano a ottenere risultati altrettanto buoni.

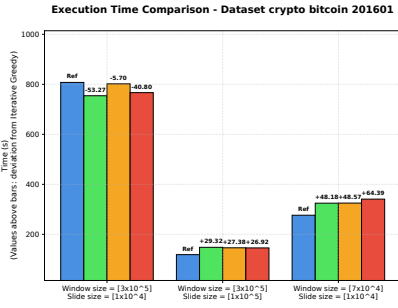


Figure 4.15: *
Tempo di esecuzione

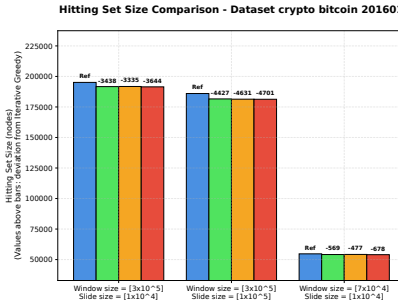


Figure 4.16: *
Dimensione media Hitting Set

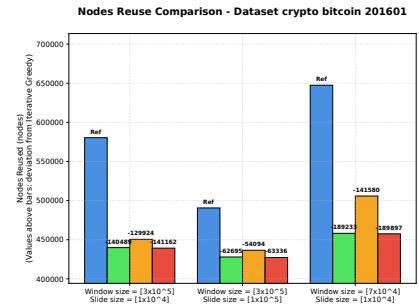


Figure 4.17: *
Numero nodi distinti scelti

— Iterative Greedy — Sliding different Greedy — Sliding New Nodes — Sliding Reuse

4.5 Analisi critica e trade-off

I risultati presentati nella sezione precedente confermano solo parzialmente le aspettative formulate in apertura del capitolo precedente. L'approccio incrementale non rappresenta un miglioramento universale rispetto alla baseline Iterative Greedy, quanto più un compromesso la cui efficacia dipende fortemente dalla struttura del dataset analizzato. All'interno della sezione seguente si discutono nello specifico i casi in cui la soluzione proposta ha rispettato le aspettative, i casi in cui le ha disattese, e si tenta di fornire una spiegazione di tali divergenze; viene inoltre analizzato il rapporto che ha la soluzione Sliding Reuse rispetto alle sue versioni Sliding different Greedy e Sliding New Nodes.

Quando l'algoritmo funziona

I casi d'uso più rappresentativi del comportamento atteso sono stati i dataset Crypto Bitcoin e Email Enron, all'interno dei quali, come predetto correttamente dalle aspettative presentate nel capitolo precedente, gli algoritmi basati su una struttura a finestra temporale a scorrimento hanno osservato dei miglioramenti in fatto di dimensione media dell'Hitting Set ($|X|_T$) e in fatto di numero di nodi distinti scelti (R) rispetto a quelli osservati dall'algoritmo iterativo greedy. Tali risultati però non sono sempre accompagnati da un analogo vantaggio in termini di tempo di esecuzione, nonostante si possa vedere un leggero vantaggio in termini di tempo di esecuzione all'interno del primo caso di test di Email Enron e di Crypto Bitcoin, per riuscire a definire un miglioramento sostanziale di T_{exec} è necessario

aumentare il numero di iperarchi e aumentare il rapporto ϵ/δ , mantenendo però una distribuzione degli iperarchi costante. Tale distribuzione di arrivo quasi costante unita alla bassa densità del dataset Crypto Bitcoin ha creato infatti le condizioni ideali affinché l'algoritmo possa esprimersi al meglio. Risulta inoltre necessario sottolineare come a prescindere dal dataset considerato, la scelta di un rapporto ϵ/δ alto ha portato a un miglioramento generale dei risultati, in quanto l'algoritmo è riuscito a sfruttare al meglio, ove possibile, il riutilizzo dei nodi attraverso la finestra temporale a scorrimento.

Quando l'algoritmo non rispetta le aspettative

Opposti ai casi d'uso presentati precedentemente, i casi d'uso più rappresentativi di un comportamento inatteso sono stati i dataset Coauth history e Contacts high school. Come previsto all'interno del capitolo precedente, a causa della non stazionarietà nella distribuzione degli arrivi degli iperarchi, in entrambi i casi si è potuto notare come l'aumento nel tempo di esecuzione totale (T_{exec}) non sia stato corrisposto da una diminuzione significativa e apprezzabile dal punto di vista della dimensione media della soluzione ($|\overline{X}|_\tau$) e del numero di nodi distinti scelti (R). Tali risultati sono motivati dal fatto che, quando le interazioni si concentrano in burst isolati o in un singolo picco terminale, la memoria storica costruita dall'algoritmo non ha modo di essere sfruttata prima che la finestra si sposti oltre, vanificando l'investimento computazionale fatto per mantenerla. Va precisato che l'effetto non risulta completamente identico nei due dataset, su Coauth history il margine di miglioramento risulta nullo o negativo in quasi tutte le configurazioni testate, mentre nel caso di Contacts high school la soluzione mantiene un vantaggio minimo ma non sempre nullo, segno che la sola intermittenza della distribuzione non è sufficiente ad annullare del tutto il beneficio del riuso, ma ne riduce significativamente l'entità rispetto ai casi favorevoli discussi in precedenza.

Confronto tra le varianti algoritmiche

I risultati ci consentono inoltre di valutare quanto le singole scelte progettuali (si veda il Capitolo 4.2) abbiano effettivamente impattato sul comportamento del sistema finale. L'analisi delle tre varianti mostra infatti che non tutte le modifiche progettuali hanno prodotto un cambiamento radicale e che l'effetto di ciascuna dipende dalla componente dell'algoritmo che viene alterata. La variante **Sliding different Greedy**, che adotta una selezione randomica dell'iperarco di riferimento invece che a cardinalità minima, mostra un comportamento discontinuo sul tempo di esecuzione a causa della sua componente randomica. Nonostante in alcuni casi di test tale algoritmo fornisca soluzioni leggermente migliori rispetto a quelle viste nell'algoritmo Sliding Reuse, i risultati sono quasi completamente comparabili, se non addirittura in media leggermente peggiori. Va peraltro riconosciuto che questa variante minima è causata dalla variazione minima che è stata applicata all'interno della stessa euristica hyperedge-oriented; un test più probante avrebbe richiesto un'euristica vertex-oriented strutturalmente distinta come quella descritta da Johnson [15] all'interno del capitolo riguardo allo stato dell'arte (si veda il Capitolo 2.2). La variante **Sliding New Node**, che inverte la priorità della funzione ReuseGreedyHittingSet preferendo nodi privi di storico, produce invece un effetto netto e coerente con l'ipotesi di progettazione. Come si può notare all'interno delle soluzioni mostrate in tutti i dataset, essa presenta un numero di nodi distinti scelti (R) peggiore, o al più comparabile, rispetto a quello denotato all'interno di Sliding Reuse, e un generale aumento nel tempo di esecuzione totale (T_{exec}). In conclusione, il confronto tra varianti suggerisce che la componente realmente determinante del sistema non sia tanto l'algoritmo greedy di base, quanto piuttosto la politica di priorità nel riuso dei nodi, la cui sola inversione è sufficiente a vanificare gran parte del beneficio in R ottenuto dalla soluzione proposta.

Il costo computazionale come contropartita sistematica

Un risultato trasversale a tutti i dataset analizzati è che il tempo di esecuzione di **Sliding Reuse** risulta quasi sempre superiore a quello di Iterative Greedy, indipendentemente dalla qualità della soluzione ottenuta. Tale risultato, in apparenza in contrasto con l'obiettivo dichiarato in fase di progettazione (si veda il Capitolo 3.2), trova spiegazione nell'analisi di complessità (si veda il Capitolo 3.4). All'interno della soluzione la funzione **Cleanup**, che nel caso pessimo richiede l'esecuzione della funzione **ReuseGreedyHittingSet** sull'intera porzione di finestra ancora scoperta, introduce dei costi computazionali non trascurabili rispetto a quelli che si trovano all'interno di un algoritmo greedy

puro, privo di strutture ausiliarie da mantenere e consultare a ogni passo. Il prezzo pagato per ottenere un minor turnover dei nodi nel tempo si traduce dunque in un aumento del lavoro richiesto per ciascun singolo aggiornamento.

Contesti d'uso e limiti dell'approccio

Analizzando l'insieme delle osservazioni precedenti emergono alcune situazioni in cui l'algoritmo riesce a sfruttare appieno il potenziale della finestra temporale a scorrimento. La soluzione proposta risulta particolarmente vantaggiosa in tutti quei casi in cui la distribuzione di arrivo degli iperarchi può essere considerata relativamente uniforme nel tempo e priva di crescite esponenziali capaci di concentrare la maggior parte degli eventi in una porzione ristretta della finestra di osservazione. Al contrario, l'algoritmo risulta poco indicato, o nei casi peggiori controproducente, in presenza di forte burstiness o di picchi concentrati di attività, come osservato nel dataset Coauth history, e in tutti i contesti nei quali il vincolo principale è la latenza di aggiornamento piuttosto che il numero di nodi distinti coinvolti nel tempo. In tali casi il costo aggiuntivo della gestione dello storico non viene ripagato da un miglioramento della qualità della soluzione, rendendo l'algoritmo iterativo greedy una scelta preferibile nonostante la sua natura non incrementale. Va infine osservato come il trade-off identificato non sia univoco nemmeno all'interno dello stesso dataset, ma dipenda anche dal rapporto tra l'ampiezza della finestra ϵ e il passo di scorrimento δ . Quando il rapporto tra le due tende a infinito, ovvero quando si considerano finestre temporali più ampie e un minor spostamento di tale finestra, la soluzione implementa attraverso l'utilizzo della finestra temporale a scorrimento risulta migliore a causa della maggiore opportunità di riutilizzo dei nodi e del minore ricalcolo necessario, mentre quando il rapporto tende a uno o a un valore minore di uno, l'utilizzo della finestra temporale a scorrimento aggiunge solamente dei costi computazionali che non è in grado di coprire. La scelta dei parametri ϵ e δ rimane quindi, anche alla luce dei risultati sperimentali, un compromesso da calibrare sul contesto applicativo specifico, piuttosto che un valore universalmente ottimale.

5 Conclusioni e sviluppi futuri

Il presente lavoro di tesi ha affrontato il problema combinatorio dell'Hitting Set in ipergrafi temporali, proponendo per esso una soluzione incrementale basata su una finestra temporale a scorrimento. L'obiettivo del progetto è stato quello di evitare il ricalcolo completo ad ogni spostamento della finestra temporale, in modo da poter ottimizzare i tempi e la dimensione della soluzione stessa, introducendo un concetto di stabilità temporale per aumentare il riutilizzo dei nodi. L'algoritmo sviluppato sfrutta il principio di stabilità temporale di una porzione delle interazioni per evitare il ricalcolo totale dell'Hitting Set. Tale soluzione è stata implementata attraverso l'utilizzo della funzione `Window Slide Function` e mediante le relative strutture di supporto come `NodePresence` per tenere traccia dei nodi visitati in precedenza. La seguente funzione è stata successivamente valutata sperimentalmente attraverso l'utilizzo di quattro dataset reali strutturalmente diversi in modo da poter analizzare i trade-off strutturali e computazionali della soluzione. L'utilizzo dell'approccio incrementale è risultato particolarmente efficace all'interno di scenari caratterizzati da distribuzioni temporali uniformi o più in generale senza concentrazioni troppo elevate, in tali contesti, l'algoritmo è risultato significativamente migliore rispetto ai classici approcci tradizionali, riducendo sia la dimensione media dell'Hitting Set che il numero totale di nodi distinti coinvolti nell'intera evoluzione della rete. Di contro, sono stati però evidenziati dei limiti sistematici legati alla natura dei dati, in presenza di distribuzioni fortemente intermittenti e concentrate, il costo computazionale richiesto per la gestione delle strutture dati ausiliarie e per le fasi di potatura delle ridondanze non viene compensato da un reale guadagno nella qualità della soluzione. Il ricalcolo della soluzione da zero, può risultare una soluzione praticabile all'interno di questi specifici scenari in quanto competitivamente preferibile in termini di tempo di esecuzione rispetto all'algoritmo sviluppato. L'analisi ha infine confermato come l'ampiezza della finestra temporale ϵ , il passo di scorrimento della finestra δ e il loro rapporto rappresentano ulteriori parametri fondamentali per riuscire a determinare l'efficacia della soluzione.

5.1 Sviluppi futuri

I limiti mostrati dai risultati ottenuti aprono la strada a diverse direzioni di ricerca futura:

- **Parallelizzazione e Computazione Distribuita:** data l'attuale implementazione in modalità single-thread, un'estensione naturale prevede lo sviluppo dell'algoritmo per architetture parallele o contesti di stream processing, parallelizzando le fasi di verifica delle ridondanze e di aggiornamento dei nodi.
- **Strategie di Epurazione Adattive della Memoria:** per mitigare l'overhead della struttura `NodePresence`, si potrebbero studiare meccanismi di decadimento temporale o politiche di rimozione basate su un fattore di dimenticanza, eliminando dalla cronologia i nodi inattivi da troppo tempo.
- **Approssimazione Garantita e Rilassamento del Problema:** infine, risulterebbe molto interessante dal punto di vista teorico analizzare formalmente il fattore di approssimazione dell'algoritmo incrementale proposto rispetto all'ottimo globale e valutarne eventuali varianti euristiche e probabilistiche.

Bibliography

- [1] Pankaj K. Agarwal, Hsien-Chih Chang, Subhash Suri, Allen Xiao, and Jie Xue. Dynamic geometric set cover and hitting set. *CoRR*, abs/2003.00202, 2020.
- [2] Alessia Antelmi, Gennaro Cordasco, Mirko Polato, Vittorio Scarano, Carmine Spagnuolo, and Dingqi Yang. A survey on hypergraph representation learning. *ACM Comput. Surv.*, 56(1):24:1–24:38, 2024.
- [3] Federico Battiston, Giulia Cencetti, Iacopo Iacopini, Vito Latora, Maxime Lucas, Alice Patania, Jean-Gabriel Young, and Giovanni Petri. Networks beyond pairwise interactions: structure and dynamics. *CoRR*, abs/2006.01764, 2020.
- [4] Thomas Bläsius, Tobias Friedrich, David Stangl, and Christopher Weyand. An efficient branch-and-bound solver for hitting set. In Cynthia A. Phillips and Bettina Speckmann, editors, *Proceedings of the 24th Symposium on Algorithm Engineering and Experiments, ALENEX 2022, Alexandria, VA, USA, January 9-10, 2022*, pages 209–220. SIAM, 2022.
- [5] Alain Bretto. *Hypergraph Theory: An Introduction*, volume 1. Springer, Mathematical Engineering; Cham, 2013.
- [6] Niv Buchbinder and Joseph Naor. The design of competitive online algorithms via a primal-dual approach. *Found. Trends Theor. Comput. Sci.*, 3(2-3):93–263, 2009.
- [7] Vasek Chvátal. A greedy heuristic for the set-covering problem. *Math. Oper. Res.*, 4(3):233–235, 1979.
- [8] Rodney G. Downey and Michael R. Fellows. *Fundamentals of Parameterized Complexity*. Texts in Computer Science. Springer, 2013.
- [9] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *J. ACM*, 45(4):634–652, 1998.
- [10] Andrew Gainer-Dewar and Paola Vera-Licona. The minimal hitting set generation problem: Algorithms and computation. *SIAM J. Discret. Math.*, 31(1):63–100, 2017.
- [11] M. R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [12] Cencetti Giulia, Battiston Federico, Lepri Bruno, and Karsai Márton. Temporal properties of higher-order interactions in social networks. *Scientific Reports*, 11(1):7028, 2021.
- [13] Fernando C. Gomes, Cláudio Nogueira de Meneses, Panos M. Pardalos, and Gerardo Valdisio R. Viana. Experimental analysis of approximation algorithms for the vertex cover and set covering problems. *Comput. Oper. Res.*, 33(12):3520–3534, 2006.
- [14] Petter Holme and Jari Saramäki. Temporal networks. *Physics Reports*, 519(3):97–125, 2012. Temporal Networks.
- [15] David S. Johnson. Approximation algorithms for combinatorial problems. *J. Comput. Syst. Sci.*, 9(3):256–278, 1974.

- [16] Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Proceedings of a symposium on the Complexity of Computer Computations, held March 20-22, 1972, at the IBM Thomas J. Watson Research Center, Yorktown Heights, New York, USA*, The IBM Research Symposia Series, pages 85–103. Plenum Press, New York, 1972.
- [17] Simon Korman. On the use of randomization in the online set cover problem. *Weizmann Institute of Science*, 2(34):7, 2004.
- [18] Quintino Francesco Lotito, Martina Contisciani, Caterina De Bacco, Leonardo Di Gaetano, Luca Gallo, Alberto Montresor, Federico Musciotto, Nicolò Ruggeri, and Federico Battiston. Hypergraphx: a library for higher-order network analysis. *J. Complex Networks*, 11(3), 2023.
- [19] Quintino Francesco Lotito and Alberto Montresor. Efficient algorithms to mine maximal span-trusses from temporal graphs. *CoRR*, abs/2009.01928, 2020.
- [20] Gary McGuire, Bastian Tugemann, and Gilles Civario. There is no 16-clue sudoku: Solving the sudoku minimum number of clues problem via hitting set enumeration. *Exp. Math.*, 23(2):190–217, 2014.
- [21] Erick Moreno-Centeno and Richard M. Karp. The implicit hitting set approach to solve combinatorial optimization problems with an application to multigenome alignment. *Operations Research*, 61(2):453–468, 2013.
- [22] Keisuke Murakami and Takeaki Uno. Efficient algorithms for dualizing large-scale hypergraphs, 2011.
- [23] Mark Newman. *Networks*. Oxford University Press, Oxford, 2 edition, 2018.
- [24] Alon Noga, Awerbuch Baruch, and Azar Yossi. The online set cover problem. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing*, STOC '03, page 100–105, New York, NY, USA, 2003. Association for Computing Machinery.

Ringraziamenti

Ora che sono arrivato alla fine di questo percorso, sento il bisogno di dedicare qualche riga a tutte quelle persone che sono state necessarie affinché la stesura di questa tesi fosse possibile e, più in generale, affinché io potessi raggiungere questo risultato.

I primi ringraziamenti vanno al mio relatore, Alberto Montresor, per il prezioso lavoro di revisione di questa tesi, e al mio co-relatore, Quintino Francesco Lotito, che mi ha seguito instancabilmente durante tutto il mio percorso di tirocinio interno. Desidero ringraziarli non solo per il supporto tecnico e professionale che mi hanno offerto, ma soprattutto perché entrambi rappresentano per me un modello da seguire, sia dal punto di vista accademico e professionale, sia da quello personale. Poter essere stato guidato da loro è stato dunque per me un vero e proprio privilegio ed onore.

Un ringraziamento speciale va ai miei genitori, rispettivamente co-co-relatore e co-co-relatrice per il supporto fornito durante lo sviluppo di questo lavoro. Grazie al loro sostegno sono riuscito a finalizzare questo percorso sentendomi sempre supportato e sostenuto anche nei momenti più incerti e difficili di questo percorso.

Infine, il ringraziamento più grande va a tutte le persone che ho incontrato e che mi hanno cambiato durante questo percorso a Trento. Citarle una per una risulterebbe difficile e al contempo riduttivo; sento però il bisogno di diversificare un minimo i ringraziamenti, poiché ogni gruppo che ho frequentato e che mi ha sostenuto ha avuto un valore diverso all'interno della mia esperienza universitaria. Vorrei prima di tutto ringraziare i gruppi che si sono formati attorno a me all'interno dello studentato Nest durante la mia permanenza a Trento. Il gruppo del primo anno e quello del terzo anno sono stati entrambi fondamentali nello sviluppo del mio percorso accademico e personale: anche se hanno contribuito in modo diverso, sono stati e continuano a essere il motivo per cui ho deciso di proseguire il mio percorso accademico. Vorrei inoltre ringraziare i miei compagni di corso: attraverso il loro supporto sono riuscito a instaurare un sano rapporto di competitività, che mi ha portato a un miglioramento personale, e al tempo stesso al ritrovamento di un gruppo di persone che condivide i miei stessi interessi e le mie stesse passioni. Infine ritengo necessario un ringraziamento personale a quelle poche persone che mi hanno accompagnato durante tutti questi tre lunghissimi anni, a voi va il mio grazie più sincero dato il supporto che mi avete dimostrato durante l'interezza di questi tre anni, grazie per essere stati la mia famiglia a 400Km da casa.